



Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: July 2026

Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures	ix
List of Tables	xi
List of Algorithms	xiii
List of Symbols	xv
List of Acronyms	xix
1 Background	1
1.1 Contributions	3
1.2 Structure of the Thesis	3
2 Preliminaries	5
2.1 Notation	6
2.2 Fully Homomorphic Encryption	7
2.2.1 The BGV Scheme	9
2.2.2 Multi-Party FHE	10
2.3 Residue Number Systems	11
2.3.1 Double CRT Polynomial	12
2.3.2 Base Conversion	13
2.4 Gadget Decomposition	15
2.4.1 RGSW	18
2.5 (Somewhat) Practical Anonymous Router	21
2.5.1 System Model	21
2.5.2 The Protocol	21
2.5.3 Requirements	23
3 Methodology	25
3.1 Code Structure	26
3.2 Implementation Details of the Core Library	27
3.2.1 RGSW Encryption	29

3.2.2	External Product	30
3.2.3	Internal Product	32
3.2.4	SIMD Batching	33
3.3	Implementation Details of sPAR	33
3.3.1	Server Write Step	33
3.4	Experimental Setup	35
4	Results and Discussion	39
4.1	Parameters	40
4.2	Noise Analysis	41
4.2.1	Core Library	41
4.2.2	sPAR	43
4.3	Performance Benchmarks	44
4.3.1	Core Library	44
4.3.2	sPAR	45
4.4	Discussions	46
4.4.1	SIMD Packing	46
4.4.2	Theoretical Hybrid Improvement	46
5	Conclusion and Future Work	51
5.1	Conclusion	51
5.2	Future Work	52
	References	53
	Appendices	
A	Number Theory	57
A.1	The Chinese Remainder Theorem	57
A.2	The Number Theoretic Transform	57
B	Noise Asymmetry in RGSW Products	59

List of Figures

3.1	Each component depends only on what lies directly beneath it. The sPAR libraries consume the core library solely through the <code>IExtendedContext</code> interface.	26
4.1	Accumulated noise from chained external products	41
4.2	Accumulated noise from chained internal products	42
4.3	Noise in L after full round.	43
4.4	Hybrid noise growth in chained external products	48

List of Tables

3.1	Primary functions declared by <code>IExtendedContext</code>	27
3.2	Common bitwise operations	28
4.1	Base parameters for BV-RNS	40
4.2	Largest number of error bits in L after 1 full round.	43
4.3	Mean CPU time in milliseconds spent per operation over T threads . . .	44
4.4	Fitted serial time β_0 , parallel work β_1 , serial fraction f and extrapolated execution time at $T = 12$	44
4.5	Measured computation time per call of the sPAR functions, 6 threads. .	45
4.6	Mean CPU time spent per operation, single-threaded.	48

List of Algorithms

2.1	Fast Base Extension	14
2.2	SCALE(x, P)	14
2.3	HomPlacing* for the choice of three slots [DP25]	22
3.1	Center	28
3.2	Modulo	28
3.3	Parallel RGSW-BV Encryption	30
3.4	Parallel Signed Decomposition	31
3.5	External Product	32
3.6	Internal Product	32
3.7	Server::Write	35
4.1	Hybrid-RGSW Encryption	48

List of Symbols

$\mathcal{B}_Q$	Residue Number System (RNS) basis $\{q_0, \dots, q_{k-1}\}$ for $Q = \prod_i q_i$.
β	Decomposition quality.
χ	Error distribution over R_Q .
$E(T)$	Execution time on T threads.
ϵ	Noise term of a ciphertext.
η	Number of clients in a sPAR round.
$\boxdot$	External product.
f	Serial fraction of a parallelized task.
Γ	Carry offset for parallel signed decomposition.
$\boxtimes$	Internal product.
$\otimes$	Kronecker product.
ℓ	Decomposition parameter.
λ	Security parameter.
$\log$	Logarithm base 2.
μ	Message of an (Ring) Gentry-Sahai-Waters (RGSW) ciphertext.

$\ \cdot\ $	Max norm $\ v\ _\infty = \max_i v_i $.
ω_i	Primitive $2N$ -th root of unity modulo q_i .
P	Auxiliary modulus, with basis $\mathcal{B}_P$ of size l .
$\hat{q}_i$	Chinese Remainder Theorem (CRT) factor Q/q_i .
R_Q	Working ring $\mathbb{Z}_Q[X]/(X^N + 1)$.
σ	Standard deviation of χ .
T	Number of threads.
w	Bit-width of a machine word.
$[x]_Q$	Centered modulo $x \in [-Q/2, Q/2)$.
$\mathbb{Z}_Q$	Integers modulo Q , centered.

List of Acronyms

CRT Chinese Remainder Theorem.

DC-net Dining Cryptographers network.

FHE Fully Homomorphic Encryption.

GSW Gentry-Sahai-Waters.

HE Homomorphic Encryption.

LWE Learning With Errors.

MCFE Multi-Client Functional Encryption.

MPC Multi-Party Computation.

NIAR Non-Interactive Anonymous Router.

NTT Number Theoretic Transform.

RGSW (Ring) Gentry-Sahai-Waters.

RLWE Ring Learning With Errors.

RNS Residue Number System.

SHE Somewhat Homomorphic Encryption.

SIMD Single Instruction, Multiple Data.

sPAR (Somewhat) Practical Anonymous Router.

Chapter 1

Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Computation (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21], which proposed using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*, without learning anything about the permutation. This makes it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

2 1. BACKGROUND

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Anonymity for recipients rests on *indistinguishability obfuscation with sub-exponential security*, a primitive guaranteeing that no efficient adversary distinguishes the obfuscations of two functionally equivalent circuits with more than sub-exponentially small advantage. Whether it can be constructed practically is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round.

These problems were taken up by the sPAR [DP25], which retains the anonymous-router functionality of NIAR but rebuilds it on FHE. A server able to compute on ciphertexts can place messages into a shuffled list without decrypting any of them, so the permutation exists only inside the computation and never in the server’s view. Every primitive the construction requires is one that established encryption schemes already provide, recipient anonymity follows from an ordinary encryption layer rather than from obfuscation, and the permutation is redrawn from client randomness in every round. The price is a second round trip, since the decryption key is split across the clients and the shuffled batch cannot be opened without them.

The idea that a server might compute on data it is unable to read is as old as public-key cryptography itself. Rivest, Adleman and Dertouzos observed that RSA is multiplicative with respect to multiplication, and asked whether a *fully* homomorphic scheme could exist [RAD78]. The question stayed open for three decades. Schemes were found that supported unbounded additions or a single multiplication, but never both without bound, as every candidate accumulated noise until its ciphertexts ceased to decrypt. Gentry settled the question in 2009 by evaluating a scheme’s own decryption circuit homomorphically, refreshing the noise of a ciphertext instead of letting it grow, and obtained the first FHE scheme [Gen09].

FHE is frequently called the holy grail of privacy-preserving computation, as it dissolves the usual trade-off between using a service and revealing data to it. For anonymous communication the appeal is more specific. An anonymous router is precisely a computation whose inputs and outputs the computing party must not learn, which is the problem FHE was built to solve. Gentry’s construction was far too slow to deploy, but the schemes that followed it have become fast enough that

the question is no longer whether such a router can be described, but what it costs to run.

That question is open for sPAR. The original paper establishes the construction and demonstrates it on a proof-of-concept built with TFHE, leaving its behaviour under a mature, exact-arithmetic scheme and its scaling in the number of participants unmeasured. This thesis closes that gap by implementing the protocol on BGV and evaluating it empirically.

1.1 Contributions

This thesis makes three contributions.

The first is a port of the TFHE-style external and internal products to the RNS variant of BGV. These products are what make long chains of encrypted binary multiplications affordable, but they are defined over a digit decomposition of ciphertext coefficients, which an RNS implementation never materializes. This construction is delivered as a library that extends OpenFHE [BAB+22] and is usable independently from sPAR. The construction is given in chapter 3.

The second is a complete implementation of sPAR on top of that library. And finally, the first empirical evaluation of the protocol beyond its original proof of concept. The implementation is described in chapter 3 and measured in chapter 4.

1.2 Structure of the Thesis

Chapter 2 introduces the building blocks the protocol rests on, covering FHE and the BGV scheme, residue number systems, gadget decomposition and the two RGSW products, before presenting sPAR itself and the requirements it places on the scheme beneath it. Chapter 3 describes the implementation of both libraries and the experimental setup used to obtain the measurements. Chapter 4 reports the parameters, the noise growth and the benchmarks, and proposes a possible improvement to the protocol. Finally, chapter 5 concludes the report and outlines the remaining work.

Chapter 2

Preliminaries

This section begins section 2.1 by establishing the common notation used throughout the thesis. Section 2.2 introduces homomorphic encryption, the BGV scheme and its multi-party variant. The following section introduces RNS, and the remainder of the section is dedicated to the sPAR construction, and the FHE primitives it depends upon.

2.1 Notation

The following conventions hold throughout this thesis. All logarithms are base 2, so that $\log x$ abbreviates $\log_2 x$. Ring elements, vectors and matrices are denoted by plain italic letters, and their type is stated at first use. For a vector v , both v_i and $v[i]$ denote the i -th component, and a sequence $(f(0), \dots, f(k-1))$ of k elements is abbreviated $\{f(i)\}_{i=0}^{k-1}$. The norm $\|\cdot\|$ is always the infinity norm $\|v\|_\infty = \max_i |v_i|$, the largest absolute value among the components of a vector or, for a polynomial, among its coefficients. Asymptotic costs are given in the usual $\mathcal{O}(\cdot)$ notation and count homomorphic operations or transmitted ciphertexts rather than machine instructions; the unit is named wherever such a bound is stated.

Definition 2.1. Centered Residues Let $\mathbb{Z}_Q = \mathbb{Z}/Q\mathbb{Z}$ denote the ring of integers modulo $Q \geq 2$. While any set of residue class representatives may stand for $\mathbb{Z}_Q$, this work canonically identifies $\mathbb{Z}_Q$ with the Q consecutive integers centered at the origin, selected by the following map.

$$[a]_Q = ((a + \lfloor Q/2 \rfloor) \bmod Q) - \lfloor Q/2 \rfloor \quad (2.1)$$

The flooring makes the map well defined for every modulus, and every representative satisfies $|[a]_Q| \leq \lfloor Q/2 \rfloor$. For example, $\mathbb{Z}_5 = \{-2, -1, 0, 1, 2\}$. Applied to a vector or polynomial, $[\cdot]_Q$ acts on each component or coefficient. The size of an element modulo Q always refers to the norm of its centered representative lifted to $\mathbb{Z}$ or $\mathbb{Z}[X]$; it is this convention that bounds quantities by $Q/2$ rather than by Q .

Definition 2.2. Working Ring All plaintext and ciphertext spaces in this work are cyclotomic quotient rings of the form $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$, where the dimension N is a power of two and Q is a modulus. An element $a \in R_Q$ is a polynomial of degree at most $N-1$ with centered coefficients, written in the **canonical form** (2.2) or the equivalent **coefficient form** (2.3).

$$a = c_0 + c_1X + \dots + c_{N-1}X^{N-1} = \sum_{i=0}^{N-1} c_i X^i \quad (2.2)$$

$$a = (c_0, c_1, \dots, c_{N-1}) = \{c_i\}_{i=0}^{N-1} \quad (2.3)$$

2.2 Fully Homomorphic Encryption

Homomorphic Encryption (HE) allows computation directly on encrypted data: an untrusted party can evaluate a function over ciphertexts without learning anything about the underlying plaintexts. With few exceptions, the best-performing schemes [BGV11; FV12; CKKS16; CGGI18] base their security on the Learning With Errors (LWE) problem introduced by Regev [Reg09]; this thesis relies on its ring variant, Ring Learning With Errors (RLWE) [LPR12].

An instantiation is fixed by a small set of parameters. The working ring R_Q of Definition 2.2 contributes the ring dimension N and the *ciphertext modulus* Q . The *plaintext modulus* $t \ll Q$ defines the plaintext space R_t , and the *message scaling factor* Δ determines where inside a ciphertext the message is placed; its value is scheme-specific. Noise is drawn from an *error distribution* χ over R_Q , a discrete Gaussian of small standard deviation σ . All parameters are chosen so that breaking the scheme requires on the order of 2^λ operations, where λ denotes the *security parameter*.

Definition 2.3. RLWE Distribution For a fixed secret $s \in R_Q$, the RLWE distribution $A_{s,\chi}$ over R_Q^2 is sampled by drawing $a \xleftarrow{\$} R_Q$ uniformly at random and $\epsilon \xleftarrow{\$} \chi$, and outputting the pair $(a \cdot s + \epsilon, a)$.

Definition 2.4. Decision RLWE Problem Given arbitrarily many independent samples from R_Q^2 , the decision RLWE problem is to distinguish whether they were drawn from $A_{s,\chi}$ for a fixed secret s or from the uniform distribution on R_Q^2 . The RLWE assumption states that no probabilistic polynomial-time adversary solves this problem with advantage better than negligible in λ [Reg09; LPR12].

The assumption is stated for an arbitrary fixed secret, leaving the distribution of s open. Implementations draw it from the ternary distribution over $\{-1, 0, 1\}^N$, which keeps $\|s\|$ minimal and thereby bounds every noise term in which the secret appears as a factor.

Definition 2.5. RLWE Ciphertext An RLWE ciphertext encrypting a plaintext $m \in R_t$ under a persistent secret key $s \xleftarrow{\$} \{-1, 0, 1\}^N$ is a pair $x = (b, a) \in R_Q^2$, obtained by drawing a fresh sample from $A_{s,\chi}$ and adding the scaled message Δm to its first component. The result consists of the **masked element** b and the **public element** a .

$$x = (b, a) = (a \cdot s + \Delta m + \epsilon, a) \quad (2.4)$$

We write $\text{RLWE}_s(m)$ for an encryption of m under s , abbreviated $\text{RLWE}(m)$ when the key is clear from context, and $\text{ENCRYPT}(m) \mapsto \text{RLWE}(m)$ for the procedure producing a fresh such ciphertext.

Definition 2.6. Decryption The function $\text{DECRYPT}(\text{RLWE}_s(m), s) \mapsto m$ recovers the plaintext from a ciphertext (b, a) by computing $b - a \cdot s = \Delta m + \epsilon$ and removing the noise ϵ . Recovery is correct only while $\|\epsilon\|$ remains below a scheme-dependent bound.

The linear operations on RLWE ciphertexts are *free*, in the sense that applying them to the ciphertext components applies them to the underlying plaintexts without any additional machinery.

Definition 2.7. Addition The sum of two ciphertexts $\text{RLWE}_s(m_x) = (b_x, a_x)$ and $\text{RLWE}_s(m_y) = (b_y, a_y)$ under the same key is the component-wise sum, satisfying $\text{RLWE}_s(m_x) + \text{RLWE}_s(m_y) = \text{RLWE}_s(m_x + m_y)$.

Correctness follows by expanding the masked elements.

$$b_x + b_y = (a_x + a_y) \cdot s + \Delta(m_x + m_y) + (\epsilon_x + \epsilon_y) \quad (2.5)$$

The sum is a well-formed encryption of $m_x + m_y$ with public element $a_x + a_y$, but it carries the combined noise $\epsilon_x + \epsilon_y$.

Definition 2.8. Scalar Multiplication The product of a ciphertext $\text{RLWE}_s(m) = (b, a)$ with a scalar $c \in R_Q$ is the component-wise product $c \cdot (b, a) = (c \cdot b, c \cdot a)$, satisfying $c \cdot \text{RLWE}_s(m) = \text{RLWE}_s(c \cdot m)$ with noise $c \cdot \epsilon$.

Message and noise are scaled together, so only multipliers with small coefficients keep the result decryptable. The product of two *ciphertexts* is not obtained this way. It is quadratic in the secret and must be relinearized back to a pair under s using dedicated evaluation keys, and it combines the two noise terms multiplicatively rather than additively [BGV11].

Definition 2.9. Public Key A public key is an encryption of zero, $pk = \text{RLWE}_s(0) = (a \cdot s + \epsilon_{pk}, a)$. It enables encryption without knowledge of s : with fresh $u \xleftarrow{\$} \{-1, 0, 1\}^N$ and $e_0, e_1 \xleftarrow{\$} \chi$, the public-key encryption of $m \in R_t$ is defined below.

$$\text{ENCRYPT}(m, pk) = u \cdot pk + (\Delta m + e_0, e_1) \quad (2.6)$$

By Definition 2.7 and Definition 2.8 the result is a well-formed ciphertext $\text{RLWE}_s(m)$ with noise $u \cdot \epsilon_{pk} + e_0 - e_1 \cdot s$, a sum of products of small elements.

Security comes at a price. Every ciphertext must carry a random noise term, and every homomorphic operation enlarges it. A ciphertext decrypts correctly only while its noise remains beneath the bound of Definition 2.6, and once that budget is spent

the plaintext is irrecoverable. The noise budget is the resource that determines how much computation a ciphertext can withstand.

Two remedies exist. Bootstrapping, due to Gentry [Gen09], evaluates the decryption function homomorphically, returning a fresh encryption of the same plaintext whose noise is that of the bootstrapping procedure rather than of the computation preceding it. Circuits of unbounded depth become evaluable, at the cost of by far the most expensive operation in any FHE scheme. The alternative is to fix parameters whose budget covers the whole computation in advance and to spend that budget as slowly as possible, for which the standard instrument is the modulus switch.

Definition 2.10. Modulus Switching Modulus switching transforms a ciphertext $x \in R_Q^2$ under the modulus Q into a ciphertext $x' = \lfloor x \cdot Q'/Q \rfloor \in R_{Q'}^2$ under a smaller modulus $Q' < Q$ that decrypts to the same plaintext, by scaling each component by Q'/Q and rounding to the nearest integer. The noise shrinks by the same factor, making the operation the primary means of controlling noise growth without bootstrapping.

2.2.1 The BGV Scheme

BGV [BGV11] is among the most mature of the RLWE-based schemes. It computes exactly over the plaintext ring R_t , so a homomorphic evaluation returns the same value as the corresponding plaintext computation with no approximation error; it batches N independent computations into the slots of a single ciphertext; and its RNS arithmetic is implemented and maintained in every major FHE library [KPZ21; BAB+22]. Where it departs from the general RLWE setting is in the placement of the message.

BGV fixes the message scale to $\Delta = 1$ and scales every noise term by the plaintext modulus instead, so that the masked element of Definition 2.5 reads $b = a \cdot s + m + t \epsilon_{\text{rlwe}}$ for a base noise term $\epsilon_{\text{rlwe}} \stackrel{\$}{\leftarrow} \chi$. Ciphertexts are nevertheless written in the general form of Definition 2.5 throughout this thesis, with the substitution $\Delta = 1$, $\epsilon = t \epsilon_{\text{rlwe}}$ left implicit.

Definition 2.11. BGV Decryption BGV decryption removes the mask with a centered reduction modulo Q , and the noise with a reduction modulo t .

$$\text{DECRYPT}((b, a), s) = \lfloor [b - a \cdot s]_Q \rfloor_t \quad (2.7)$$

The inner reduction recovers $m + t \epsilon_{\text{rlwe}}$ exactly over the integers provided $\|m + t \epsilon_{\text{rlwe}}\| \leq \lfloor Q/2 \rfloor$, and the outer reduction then strips the noise term and returns m . This is the concrete instance of the bound in Definition 2.6, which places decryption failure at $Q/2$.

Definition 2.12. Single Instruction, Multiple Data (SIMD) Packing

If the plaintext modulus is a prime satisfying $t \equiv 1 \pmod{2N}$, a plaintext may be reinterpreted as a vector of N independent **slots** $[c_i]_t$ on which homomorphic operations act element-wise [SV11]. For example, the slot-wise product gives $(1, 2) \cdot (2, 3) = (2, 6)$, in contrast to the polynomial product $(1+2X)(2+3X) = 2+7X+6X^2$.

The analogy to hardware SIMD extends only to the semantics, not to the cost. A slot-wise operation is an ordinary ciphertext operation and takes the time of one, whether one slot is occupied or all N are. Packing therefore buys throughput and never latency, and it improves an application only to the extent that the application has N independent computations to offer.

2.2.2 Multi-Party FHE

When mutually distrusting parties compute on shared data, no single party can be allowed to hold the secret key. Threshold variants of FHE remove this single point of trust by distributing both key generation and decryption among the parties, at the cost of an interactive decryption round. The n -out-of- n variant described below is the one implemented in OpenFHE [BAB+22], where the joint secret is the plain sum of the individual shares.

During a one-time setup, each party i samples its own secret share s_i and publishes a public share formed against the common public element a carried by the preceding party's key. By Definition 2.7 the shares accumulate into a single joint public key (Definition 2.9) for the joint secret $s = \sum_i s_i$, under which every party encrypts. The joint secret is never reconstructed, and any single honest share suffices to keep it hidden.

Decryption proceeds in two rounds. Each party first releases a *partial decryption* $d_i = a \cdot s_i + \epsilon_i^{\text{fl}}$ of the ciphertext (b, a) , computed from its own share and a fresh *flooding* term. The flooding noise ϵ_i^{fl} is drawn from a distribution much wider than χ , so that d_i statistically hides s_i . Because decryption is linear in the secret, the partial decryptions then combine into $b - \sum_i d_i = \Delta m + \epsilon - \sum_i \epsilon_i^{\text{fl}}$ without any party learning the joint secret. Correctness requires only that the flooding noise, like any other, stays within the noise budget of Definition 2.6, which is what makes the width of the flooding distribution a parameter of the scheme rather than a free choice.

2.3 Residue Number Systems

In BGV, BFV and CKKS the ciphertext modulus Q is typically large, much larger than a machine word or register (64 bits on modern computers). Numbers that do not fit inside a register must be handled with multi-precision arithmetic, which is known to be slow due to its serial nature. High-performance instantiations remedy this by using a RNS where large numbers are represented as a set of smaller, independent residues via the CRT (detailed in appendix A.1). RNS variants of BGV, BFV and CKKS are the standard in practice; OpenFHE, for example, only supports the RNS versions of these schemes [BAB+22].

Definition 2.13. RNS Basis Let w denote the bit-width of a standard machine word. A positive integer q is termed **small** if it fits entirely within this word, such that $0 < q < 2^w$. A set of k moduli $\mathcal{B}_Q = \{q_0, q_1, \dots, q_{k-1}\}$ is considered an **RNS basis** for $Q = \prod_{i=0}^{k-1} q_i$ if its k elements are all small and pairwise coprime ($\gcd(q_i, q_j) = 1$ for all $i \neq j$).

Definition 2.14. RNS Polynomial A polynomial $a \in R_Q$ with coefficients c_j is an **RNS polynomial** if it is decomposed into k polynomial **limbs** with coefficients in the basis $\mathcal{B}_Q$ via CRT, satisfying the explicit ring isomorphism:

$$R_Q \cong R_{q_0} \times R_{q_1} \times \dots \times R_{q_{k-1}} \quad (2.8)$$

The limbs are also colloquially termed **towers** or **residues**. We use the notation $[a]_{\mathcal{B}_Q}$ to denote an RNS polynomial $a \in R_Q$ represented entirely by its limbs.

Definition 2.15. First RNS form We define the **RNS form** of a as the $k \times N$ matrix (2.9), where the i -th row contains the coefficients $[c_j]_{q_i}$ of the i -th limb.

$$\begin{pmatrix} a \bmod q_0 \\ a \bmod q_1 \\ \vdots \\ a \bmod q_{k-1} \end{pmatrix} = \begin{pmatrix} c_0 \bmod q_0 & c_1 \bmod q_0 & \dots & c_{N-1} \bmod q_0 \\ c_0 \bmod q_1 & c_1 \bmod q_1 & \dots & c_{N-1} \bmod q_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod q_{k-1} & c_1 \bmod q_{k-1} & \dots & c_{N-1} \bmod q_{k-1} \end{pmatrix} \quad (2.9)$$

Because the RNS moduli are small, every coefficient in (2.9) fits within a single machine word. Furthermore, the coprimality of the basis ensures this representation is an isomorphism (2.8), allowing global reconstruction back to R_Q via CRT. Consequently, addition and subtraction of two RNS polynomials in the same basis is functionally equivalent to the element-wise addition and subtraction of their RNS forms modulo q_i . No carries propagate between the towers, allowing all k residues to be processed in parallel utilizing fast, native hardware instructions.

2.3.1 Double CRT Polynomial

While the RNS form perfectly isolates polynomial addition into word-sized registers, standard polynomial multiplication in R_Q operating directly on coefficients scales poorly at $\mathcal{O}(N^2)$. To achieve practical performance, high-speed implementations map these polynomials into an evaluation domain using the Number Theoretic Transform (NTT) described in more detail in appendix A.2.

Definition 2.16. NTT-Friendly RNS Basis Let N denote the dimension of the polynomial ring R_Q , where N is a power of two. A small modulus q_i is termed **NTT-friendly** if $q_i \equiv 1 \pmod{2N}$, guaranteeing the existence of a primitive $2N$ -th root of unity $\omega_i \in \mathbb{Z}_{q_i}$. An RNS basis $\mathcal{B}_Q$ is NTT-friendly if it consists entirely of NTT-friendly moduli.

Definition 2.17. Second RNS Form (Evaluation Form) Let $a \in R_Q$ be an RNS polynomial over an NTT-friendly basis $\mathcal{B}_Q$. The NTT is an invertible mapping that transitions a polynomial from its coefficient representation into a point-value representation (see appendix A.2).

The **evaluation form** (or **NTT form**) of a , denoted $\text{NTT}(a)$, is the $k \times N$ matrix obtained by independently applying the N -point NTT to each of its k limbs:

$$\text{NTT}(a) \equiv \begin{pmatrix} \text{NTT}_{q_0}(a \bmod q_0) \\ \text{NTT}_{q_1}(a \bmod q_1) \\ \vdots \\ \text{NTT}_{q_{k-1}}(a \bmod q_{k-1}) \end{pmatrix} \quad (2.10)$$

The evaluation form reduces polynomial multiplication in R_Q to the element-wise product of their respective matrices. The evaluation form also inherits the element-wise addition and subtraction from the RNS form, meaning an entire sequence of arithmetic operations can be executed entirely in parallel using native hardware instructions, without ever leaving the NTT domain.

However, this efficiency dictates a strict computational trade-off. Certain arithmetic operations fundamentally cannot be performed in the evaluation form and require the polynomial's coefficients. The transition between representations necessitates a Forward or Inverse NTT, which is a computationally expensive $\mathcal{O}(N \log N)$ procedure. Consequently, a primary objective in the implementation of RNS-variants of homomorphic encryption schemes is the strategic minimization of transitions between the RNS form and evaluation form.

2.3.2 Base Conversion

One central operation that is contentious in RNS is *base conversion*, where the goal is to convert an evaluation form polynomial in the basis $\mathcal{B}_Q$ to a different basis $\mathcal{B}_{Q'}$. When $Q < Q'$, the operation is called *base extension*. In this section we briefly introduce four algorithms that significantly improve the performance of base conversion in RNS, summarized in the table below.

Operation	Computes	Error
FASTB CONV	$[x]_{\mathcal{B}_Q} \mapsto [x + \epsilon]_{\mathcal{B}_P}$	$\alpha_x Q$ for $\alpha_x \in [0, k)$
FASTBASEEXTEND	$[x]_{\mathcal{B}_Q} \mapsto [x + \epsilon]_{\mathcal{B}_{QP}}$	$\alpha_x Q$ for $\alpha_x \in [0, k)$
SCALE	$[x]_{\mathcal{B}_Q} \mapsto [xP]_{\mathcal{B}_{QP}}$	0
APPROXMODDOWN	$[x]_{\mathcal{B}_{QP}} \mapsto [\lfloor x/P \rfloor - \alpha]_{\mathcal{B}_Q}$	$\alpha \in [0, k)$

Fast Base Conversion

Introduced by Bajard et al. in [BEHZ16, Equation 2], *fast base conversion* is an approximation that computes $[x]_{Q'} + \alpha_x Q$ for some integer $\alpha_x \in [0, k)$ using the RNS form residues $[x]_{\mathcal{B}_Q}$. Computing $\alpha_x Q$ is possible, but requires expensive operations in RNS and is typically skipped. Let $\hat{q}_i = Q/q_i$, then the fast base conversion is:

$$\text{FASTB CONV}(x, \mathcal{B}_Q, \mathcal{B}) = \left\{ \left[\sum_{i=0}^{k-1} [x_i \hat{q}_i^{-1}]_{q_i} \cdot \hat{q}_i \right]_b \right\}_{b \in \mathcal{B}} \quad (2.11)$$

Remark 2.18. The coefficients $\hat{q}_i$ and $\hat{q}_i^{-1}$ are the CRT factors from appendix A.1 and Remark 2.24, and can be pre-calculated to maximize the runtime performance of (2.11).

Fast Base Extension

Let P be an auxiliary modulus formed by a disjoint RNS basis $\mathcal{B}_P$ of size l , such that $P = \prod_{j=0}^{l-1} p_j$ and $\gcd(Q, P) = 1$. The extended basis is defined as the union $\mathcal{B}_{QP} = \mathcal{B}_Q \cup \mathcal{B}_P$. If we use (2.11) to convert $\mathcal{B}_Q \mapsto \mathcal{B}_{QP}$, the error $\alpha_x Q \bmod QP$ disappears in the q_i -limbs since $Q = \prod_i q_i$ is a product of every q_i .

$$x + \alpha_x Q \equiv x + \alpha_x \prod_j q_j \equiv x + q_i \cdot (\alpha_x \prod_{j \neq i} q_j) \equiv x \pmod{q_i}$$

Restricting base extension to $\mathcal{B}_Q \mapsto \mathcal{B}_{QP}$ therefore allows us to skip the k first moduli and calculate only the l p_i -limbs using $\text{FASTB CONV}(x, \mathcal{B}_Q, \mathcal{B}_P)$, appending the results to $[x]_{\mathcal{B}_Q}$.

Algorithm 2.1 Fast Base Extension

Input: $\text{NTT}([a]_{\mathcal{B}_Q})$ and an RNS basis $\mathcal{B}_P$ **Output:** $\text{NTT}([a + \epsilon]_{\mathcal{B}_{QP}})$ with $\|\epsilon\| \leq k \cdot Q/2$ 1: $a_Q \leftarrow \text{INVERSENTT}(a)$ 2: $a_P \leftarrow \text{FASTBCONV}(a_Q, \mathcal{B}_Q, \mathcal{B}_P)$ 3: **return** $a \cup \text{NTT}(a_P)$ $\triangleright$ NTT is applied to each p_i -limb individually

Scale

Observe that if $x + \alpha_x Q \bmod QP$ was multiplied by P , the error term $\alpha_x QP$ would disappear entirely. Further, $xP \equiv 0 \bmod p_j$ and the only remaining non-zero calculation $xP \bmod q_i$ can be calculated in the evaluation domain. This gives rise to a specialized $\text{SCALE}(x, P)$ operation mapping $[x]_{\mathcal{B}_Q} \mapsto [xP]_{\mathcal{B}_{QP}}$ exactly and without using any inverse or forward NTT operations.

Algorithm 2.2 $\text{SCALE}(x, P)$

Input: $\text{NTT}([x]_{\mathcal{B}_Q})$ and an RNS basis $\mathcal{B}_P$ **Output:** $\text{NTT}([xP]_{\mathcal{B}_{QP}})$ 1: **for** $(x_i, q_i) \in x$ **do** $\triangleright$ Multiply each limb by P in the evaluation domain2: $x_i \leftarrow x_i \cdot P \bmod q_i$ 3: **end for**4: $x \leftarrow x \cup \{0\}_{j=0}^{l-1}$ $\triangleright$ Append l empty limbs5: **return** x

Approximate Mod Down

Following homomorphic evaluation in the extended basis $\mathcal{B}_{QP}$, the polynomial state must be scaled back down by P to resolve the precision extension and return to the base coefficient ring R_Q . This operation, termed *Approximate Mod Down*, functionally computes a rounded rational division $\lfloor x/P \rfloor$ natively within the integer RNS domain.

The procedure exploits the fact that the exact remainder of the division, $x \bmod P$, is inherently embedded in the auxiliary limbs $[x]_{\mathcal{B}_P}$. Because basis conversion necessitates the coefficient representation, these l auxiliary limbs are first transitioned via an inverse NTT. By projecting them back into $\mathcal{B}_Q$ using FASTBCONV , we obtain an approximated remainder $r = (x \bmod P) + \alpha P$ carrying a bounded structural error $\alpha \in [0, l)$. This projected remainder is then transformed back into the evaluation domain via a forward NTT. Subtracting this evaluation-form remainder from the original q_i -limbs isolates a polynomial that is mathematically guaranteed to be a strict multiple of P . The discrete division is finally resolved by scaling the state by the modular inverse $P^{-1} \pmod{q_i}$ and severing the auxiliary basis entirely.

2.4 Gadget Decomposition

Brakerski and Vaikuntanathan [BV11] first proposed decomposing ciphertext elements into small digits as a means of controlling the noise in their *relinearization* technique. The idea has since been generalized into the idea of *gadget decomposition* and been employed in the design of numerous FHE schemes [BGV11; Bra12; FV12; GSW13; DM14; CKKS16; CGGI18].

Definition 2.19. Digit Decomposition Given an integer $\gamma \in \mathbb{Z}_q$ for a modulus $q \geq 2$, base B and decomposition parameter ℓ , the digit decomposition $\text{Decomp}(\gamma) = (\gamma_0, \gamma_1, \dots, \gamma_{\ell-1}) \in \mathbb{Z}_B^\ell$ outputs an ordered list of ℓ base- B **digits** satisfying

$$\gamma = \gamma_0 \frac{q}{B} + \gamma_1 \frac{q}{B^2} + \dots + \gamma_{\ell-1} \frac{q}{B^\ell} = \sum_{i=0}^{\ell-1} \gamma_i \frac{q}{B^{i+1}}$$

Digit decomposition has two important properties. First, if the digits γ_i of $\text{Decomp}(\gamma)$ are small, then $\langle \text{Decomp}(\gamma), \epsilon \rangle \leq \ell \cdot \max\{\gamma_i \cdot \epsilon_i\} \leq \ell B \epsilon \ll \gamma \epsilon$. This is the property that allows some control over the noise growth. Second, two integers can be multiplied through the decomposition. By construction, $\langle \text{Decomp}(\gamma), g \rangle = \gamma$ for the fixed vector $g = (q/B, \dots, q/B^\ell)$, meaning $\langle \text{Decomp}(\gamma), g \cdot x \rangle = \gamma \cdot x$ for an arbitrary integer $x \in \mathbb{Z}$. For either of these properties, the particular form of g is unimportant as long as the digits γ_i remain small. Furthermore, as we are generally working with some intrinsic error in FHE, [CGGI18] proposed allowing some error ϵ in the reconstruction, giving the definition from their paper verbatim:

Definition 2.20. (Abstract) Gadget Decomposition An efficient algorithm $\text{Decomp}^{g, \beta, \epsilon}(v)$ is a valid decomposition on the gadget $g \in \mathbb{Z}_q^\ell$ with quality $\beta \in \mathbb{Z}^+$ and precision $\epsilon \in \mathbb{Z}^+$ if and only if for any ring element $v \in R_Q$ it efficiently and publicly outputs a small vector $u \in \mathbb{Z}_q^\ell$ such that $\|u\|_\infty \leq \beta$ and $\langle u, g \rangle \leq v + \epsilon$.

The entries of the output vector u are again called *digits*, though for a general gadget they need not be digits of any base. It is convenient to name the two sides of the reconstruction rather than to carry an algorithm and a fixed gadget separately. Writing $D(v)$ for the output of $\text{Decomp}^{g, \beta, \epsilon}(v)$ and $P(u)$ for the gadget-scaled vector $g \cdot u$, the condition $\langle u, g \rangle \leq v + \epsilon$ becomes a statement about a pair of vectors, which is the form used throughout this thesis.

Definition 2.21. Gadget Property Two vectors $P(x) \in \mathbb{Z}_Q^\ell$ and $D(y) \in \mathbb{Z}_\beta^\ell$ are said to have the *gadget property* if and only if they satisfy (2.12) for any pair of ring elements $(x, y) \in R_Q^2$ with precision $\epsilon \in \mathbb{Z}_q^+$ and quality β where $\|D(y)\|_\infty \leq \beta$. If $\epsilon = 0$ then $P(x)$ and $D(y)$ are said to have the *exact* gadget property.

$$\langle P(x), D(y) \rangle = x \cdot y + \epsilon \pmod{Q} \quad (2.12)$$

Lemma 2.22. Layered Gadget Decomposition *Let (P_1, D_1) and (P_2, D_2) be two pairs with the gadget property, of quality β_1, β_2 and precision ϵ_1, ϵ_2 respectively. Their composition, defined component-wise by $(P_1 \circ P_2)(u) = P_1(P_2(u))$ and $(D_1 \circ D_2)(v) = D_1(D_2(v))$, again has the gadget property, with quality β_1 and precision $\epsilon_1 + \epsilon_2$.*

$$\langle (P_1 \circ P_2)(u), (D_1 \circ D_2)(v) \rangle = u \cdot v + \epsilon_1 + \epsilon_2 \pmod{Q} \quad (2.13)$$

Proof. Apply the gadget property of the outer pair to the vectors produced by the inner one, then the gadget property of the inner pair.

$$\langle P_1(P_2(u)), D_1(D_2(v)) \rangle = \langle P_2(u), D_2(v) \rangle + \epsilon_1 = u \cdot v + \epsilon_2 + \epsilon_1 \pmod{Q}$$

The digits of the composition are those produced last, by D_1 , so the quality is the quality β_1 of the outer decomposition. $\square$

RNS Instantiation

Digit decomposition cannot be applied directly to ring elements in an RNS scheme where the system only has access to the residues. When Bajard et al. implemented the first RNS version of the BV scheme [BEHZ16] they exchanged digit decomposition for an RNS decomposition, exploiting the fact that this decomposition is already required by the representation. Halevi, Polyakov and Shoup later simplified their construction into the form below [HPS18], which is the one used here.

Definition 2.23. HPS Gadget Recall from subsection 2.3.2 the factors $\hat{q}_i = Q/q_i$, and let $[\hat{q}_i^{-1}]_{q_i}$ denote the modular inverse of $\hat{q}_i$ modulo its own prime, so that $\hat{q}_i \cdot [\hat{q}_i^{-1}]_{q_i} \equiv 1 \pmod{q_i}$. By formulating the CRT as an inner product (A.2), it is immediately clear that the RNS decomposition with moduli q_i is an exact decomposition on $g = (\hat{q}_0 \cdot [\hat{q}_0^{-1}]_{q_0}, \dots, \hat{q}_{k-1} \cdot [\hat{q}_{k-1}^{-1}]_{q_{k-1}})$ with $\beta = \max_i \{q_i\}/2$ (assuming centered residues).

$$P_{\text{HPS}}(a) = (a \cdot [\hat{q}_0^{-1}]_{q_0} \cdot \hat{q}_0, \dots, a \cdot [\hat{q}_{k-1}^{-1}]_{q_{k-1}} \cdot \hat{q}_{k-1})_Q \quad (2.14)$$

$$D_{\text{HPS}}(a) = ([a]_{q_0}, [a]_{q_1}, \dots, [a]_{q_{k-1}}) \quad (2.15)$$

Remark 2.24. As RNS stores the polynomial $a \in R_Q$ as the set of residues $[a]_{q_i}$ for all $q_i \in \mathcal{B}_Q$, the $D_{\text{HPS}}(\cdot)$ vector (2.15) is *free* in the sense that calculating $D_{\text{HPS}}(a) \cong a$ requires no arithmetic. In fact, neither does (2.14) since $\hat{q}_i = Q/q_i = (\prod_{l=0}^{k-1} q_l)/q_i$ contains q_j as a factor unless $i = j$, in which case $\hat{q}_i \cdot [\hat{q}_i^{-1}]_{q_i} = 1$ by definition!

$$\begin{aligned} a \cdot [\hat{q}_i^{-1}]_{q_i} \cdot \hat{q}_i &= \{(a \cdot [\hat{q}_i^{-1}]_{q_i} \cdot \hat{q}_i) \bmod q_j\}_{j=0}^{k-1} \\ (a \cdot [\hat{q}_i^{-1}]_{q_i} \cdot \hat{q}_i) \bmod q_j &= \begin{cases} 0, & i \neq j \\ [a]_{q_j}, & i = j \end{cases} \end{aligned}$$

While the HPS gadget is cheap, its quality $\beta = \max_i \{q_i\}/2$ is too large for many applications, as the digits are the full residues themselves. A second layer of decomposition brings the quality down. Every digit of $D_{\text{HPS}}(a)$ fits inside a machine word, since the RNS primes are chosen that way, so the digit decomposition of Definition 2.19 applies to each limb directly and Lemma 2.22 makes the composition a valid gadget in its own right. Taking the second layer on the powers of a base B , that is on $g = (1, B, \dots, B^{\ell-1})$, splits each of the k residues into ℓ digits. The composed gadget produces $k\ell$ digits of quality $\beta = B/2$ once they are centered, and it is exact whenever $B^\ell \geq \max_i \{q_i\}$, which fixes the decomposition parameter at $\ell = \lceil \log_B \max_i \{q_i\} \rceil$.

2.4.1 RGSW

The gadget techniques above are used in the Gentry-Sahai-Waters (GSW) scheme [GSW13] and its ring variants [DM14; CGGI18], where the ciphertext itself is built around a gadget. GSW came with a homomorphic product between two of its ciphertexts, and TFHE later added an asymmetric product between an RLWE and an RGSW ciphertext [CGGI16; CGGI18]. This subsection defines the RGSW ciphertext and both products.

Definition 2.25. Gadget Matrix A gadget $g \in \mathbb{Z}_Q^\ell$ is extended to k -dimensional ciphertexts by the block-diagonal gadget matrix $G = I_k \otimes g \in R_Q^{k\ell \times k}$.

Definition 2.26. RGSW Ciphertext Given a gadget matrix $G = I_2 \otimes g$, let $Z \in R_Q^{2\ell \times 2}$ be a matrix whose 2ℓ rows are fresh encryptions $\text{RLWE}(0)$. An RGSW encryption of $\mu \in R_Q$ is

$$C = Z + \mu \cdot G \in R_Q^{2\ell \times 2} \quad (2.16)$$

An RGSW ciphertext encodes the gadget-scaled messages μg_i row by row. In the top block, μg_i is added to the masked element, replacing the zero message of the underlying $\text{RLWE}(0)$; in the bottom block it is added to the public element, so the row decrypts to $-\mu g_i \cdot s$ instead. Every row of C is therefore itself an RLWE ciphertext.

Two RGSW ciphertexts add homomorphically, just as RLWE ciphertexts do: the zero encryptions and the gadget terms add component-wise, so $C_A + C_B$ encrypts $\mu_A + \mu_B$. Addition alone is, however, not sufficient for FHE. The multiplication mirrors the vector-matrix relationship between the two ciphertext types: the RLWE ciphertext $x = (x_0, x_1)$ is mapped by a two-dimensional decomposition $G^{-1}(x)$ to a row vector of 2ℓ small ring elements, which multiplies the $2\ell \times 2$ matrix C . The only requirement on $G^{-1}(x)$ is that it approximately inverts the G matrix:

$$G^{-1}(x) \cdot G = x + e \pmod{Q} \quad (2.17)$$

with quality $\|G^{-1}(x)\| \leq \beta$. No particular decomposition is prescribed. The canonical choice is the base- B digit decomposition of the ciphertext coefficients [CGGI18], and it is the one usually meant when the external product is written down, but nothing in the definition depends on it. Any $G^{-1}(x)$ meeting (2.17) at quality β yields a valid product, and the noise bounds below are stated in terms of β alone.

Definition 2.27. External Product Let $x \in R_Q^2$ be an RLWE encryption of $m \in R_t$, let $C \in R_Q^{2\ell \times 2}$ be an RGSW encryption of $\mu \in R_Q$ under the same secret key, and let G^{-1} be a gadget decomposition satisfying (2.17). The external product

is defined as

$$\begin{aligned} \boxtimes : \text{RLWE} \times \text{RGSW} &\longrightarrow \text{RLWE} \\ (x, C) &\longmapsto x \boxtimes C = G^{-1}(x) \cdot C, \end{aligned} \quad (2.18)$$

which returns an encryption of the product of the messages: $\text{RLWE}(m) \boxtimes \text{RGSW}(\mu) = \text{RLWE}(m \cdot \mu)$. Correctness follows by inserting (2.16) into (2.18):

$$G^{-1}(x) \cdot C = G^{-1}(x) \cdot Z + \mu \cdot (x + e) \quad (2.19)$$

where $\mu \cdot x$ carries the message and the remaining terms contribute only noise; formal correctness and noise bounds are given in [CGGI18].

Lemma 2.28. Gadget Matrix Construction *Two vectors $P(u), D(v)$ that satisfy the gadget property (2.12) define a valid pair of gadget matrices with $g = P(1)$:*

$$G = I_2 \otimes P(1) \quad (2.20)$$

$$G^{-1}(x) = [D(x_0) | D(x_1)] \quad (2.21)$$

satisfying (2.17), with $\|G^{-1}(x)\| \leq \beta$ inherited from the quality of D .

Proof. The block structure reduces the product to one inner product per component, each an instance of (2.12):

$$\begin{aligned} G^{-1}(x) \cdot G &= [D(x_0) | D(x_1)] \begin{pmatrix} P(1) & 0 \\ 0 & P(1) \end{pmatrix} \\ (\langle D(x_0), P(1) \rangle, \langle D(x_1), P(1) \rangle) &= (x_0 + e_0, x_1 + e_1) = x + e \pmod{Q} \end{aligned}$$

□

Remark 2.29. The two noise terms of (2.19) behave very differently, as detailed in Appendix B. The term $G^{-1}(x) \cdot Z \leq (a \cdot s + \beta \sum_{i=0}^{2\ell-1} \epsilon_i, a)$ amplifies the masking errors of the RGSW operand by the decomposition quality. The precision term e is scaled by μ , which is why μ is typically constrained to $\{0, 1\}$ when an inexact gadget is used. The error of the RLWE operand, by contrast, is never amplified: it is carried into the result scaled only by μ . This asymmetry was observed early on [BV13; AP14; DM14], and makes GSW-style ciphertexts particularly suited to evaluating long chains of products. The noise stays low as long as the right operand is fresh, or at least has a small $\|\epsilon\|_\infty$, regardless of the error already accumulated on the left.

The external product leaves the RGSW operand untouched and returns an RLWE ciphertext, so it cannot be chained: its output is not of the right type to serve as either operand again. The original GSW construction therefore defined its product directly between two of its own ciphertexts [GSW13]. That product needs no separate machinery here. Every row of an RGSW ciphertext is an RLWE ciphertext, so applying the external product row by row multiplies the whole matrix by the same message and returns an RGSW ciphertext again.

Definition 2.30. Internal Product Let $A, C \in R_Q^{2\ell \times 2}$ be RGSW encryptions of $\mu_A, \mu_C \in R_Q$ under the same secret key and gadget, and let A_i denote the i -th row of A . The **internal product** [CGGI18] applies the external product to each row of the left operand:

$$\boxtimes : \text{RGSW} \times \text{RGSW} \longrightarrow \text{RGSW}$$

$$(A, C) \longmapsto A \boxtimes C = \begin{bmatrix} A_0 \boxtimes C \\ A_1 \boxtimes C \\ \vdots \\ A_{2\ell-1} \boxtimes C \end{bmatrix}, \quad (2.22)$$

which returns an encryption of the product of the messages: $\text{RGSW}(\mu_A) \boxtimes \text{RGSW}(\mu_C) = \text{RGSW}(\mu_A \cdot \mu_C)$.

One internal product amounts to 2ℓ external products, and correctness follows row by row: each external product multiplies the content of A_i by μ_C , so the stacked result is again of the form (2.16) with message $\mu_A \cdot \mu_C$.

The asymmetry of Remark 2.29 applies to each of the 2ℓ rows, and dictates the order in which a chain of products should be evaluated. The noise of the right-hand operand is amplified by the decomposition in every row of the result, while the noise of the left-hand operand is only carried through. When accumulating a running product, the accumulator must therefore be kept on the left with each fresh ciphertext supplied on the right, so that the noise grows additively rather than multiplicatively.

Remark 2.31. Operand Order The convention above places the RLWE operand of the external product on the left and the RGSW operand on the right, so that both products read left to right with the accumulator first. TFHE writes the external product the other way round [CGGI18], with the RGSW ciphertext on the left. The operation is the same, but any expression carried over from that literature must be transposed before it is read against (2.18).

2.5 (Somewhat) Practical Anonymous Router

sPAR realizes the anonymous router of chapter 1 from the primitives of the preceding sections [DP25]. Each of the η clients encrypts its message together with a randomly chosen destination index, and the server writes every message into an encrypted list at the position its sender picked. Both the list and the indices remain encrypted throughout the write.

2.5.1 System Model

The protocol runs in rounds between η clients and a single server. Every client submits exactly one message per round, sending a random message if it has nothing to say, and all traffic passes through the server. The server is untrusted for privacy but trusted for availability, and may collude with any number of corrupted clients. Sender anonymity follows from the IND-CPA security of the underlying scheme, with definitions and proofs given in the original paper [DP25].

Two further assumptions are what the protocol takes explicit measures to satisfy. Messages are assumed to be uniformly distributed, which is what lets placement at a random encrypted index stand in for a shuffle. Messages sharing a bucket must additionally be sorted obliviously before the list is opened, since their relative order would otherwise reveal which sender wrote first.

2.5.2 The Protocol

A one-time setup is followed by polynomially many rounds under the same keys, each consisting of four computations separated by two client-server round trips.

Setup

The clients agree on a set of FHE parameters and run the multi-party key generation of subsection 2.2.2, producing a joint public key under which every round is encrypted.

Client: Encryption

Client i encrypts its message under the joint key and samples three bucket indices $a_0, a_1, a_2 \xleftarrow{\$} [0, \eta)$ uniformly, encrypting the $\log \eta$ bits of each index separately. A single index per client would collide too often, whereas with η buckets of three slots the probability that all nine candidate slots are taken falls below 10^{-11} , and on that event the message is dropped and retransmitted in a later round [DP25].

Server: Write

The server keeps a message matrix L of $\eta \times 3$ RLWE slots and an availability matrix I of $\eta \times 3$ encrypted flags marking which slots are free. For each client it evaluates Algorithm 2.3, where a binary tree of depth $\log \eta$ turns each encrypted index into an indicator vector over the buckets, and the write loop then places the message in the first free slot among the three chosen buckets, tracked by the encrypted flag `hasWritten`. Clients upload RLWE ciphertexts only, as the index bits are expanded to RGSW by the server [CCR19; CDPP22].

Once all η messages are written, the slots of each bucket are sorted obliviously and L is broadcast to the clients.

Algorithm 2.3 HomPlacing* for the choice of three slots [DP25]

Input: Encrypted value V_r

Input: Encrypted indices A_0, A_1, A_2 , where A_d is $\log \eta$ encrypted bits $\{c_{d,0}, \dots, c_{d,\log \eta - 1}\}$

Input: Server state matrices $L_{\eta \times 3}$ and $I_{\eta \times 3}$

Output: Encrypted boolean indicating whether the value was placed

```

1:  $z_{i,d} := 0$  for  $i \in [0, \eta)$ ,  $d \in \{0, 1, 2\}$ 
2: for  $d \leftarrow \{0, 1, 2\}$  do
3:    $b_0 := 1$  and  $b_i := 0$  for  $i \in [1, 2\eta - 1)$  ▷ Empty binary tree with  $\eta$  leaves
4:   for  $i \leftarrow [0, \log \eta)$  do
5:     for  $j \leftarrow [0, 2^i)$  do
6:        $r := b_{2^i - 1 + j}$ 
7:        $b_{2^{i+1} - 1 + 2j} := r - r \cdot c_{d,i}$ 
8:        $b_{2^{i+1} + 2j} := r \cdot c_{d,i}$ 
9:     end for
10:  end for
11:   $z_{i,d} := b_{\eta - 1 + i}$  for  $i \in [0, \eta)$  ▷ Leaves: indicator vector of  $a_d$ 
12: end for
13: hasWritten := 0
14: for  $d \leftarrow \{0, 1, 2\}$  do
15:   for  $k \leftarrow \{0, 1, 2\}$  do ▷ Slots per bucket
16:     for  $i \leftarrow [0, \eta)$  do
17:        $h := z_{i,d} \cdot I_{i,k} \cdot (1 - \text{hasWritten})$ 
18:        $L_{i,k} := L_{i,k} + h \cdot V_r$ 
19:        $I_{i,k} := I_{i,k} - h$  ▷ Mark the slot as used
20:       hasWritten := hasWritten +  $h$ 
21:     end for
22:   end for
23: end for
24: return hasWritten

```

Client: Partial Decryption

Each client partially decrypts all 3η slots with its own share (subsection 2.2.2) and sends the result back to the server.

Server: Final Decryption

The server aggregates the partial decryptions of each slot (subsection 2.2.2) and recovers the list. Empty slots decrypt to zero and are discarded, leaving the η messages in an order that is a fresh uniform permutation of the senders.

2.5.3 Requirements

Algorithm 2.3 is stated for a generic homomorphic multiplication, but the circuit it describes is deep. Every level of the binary tree multiplies the running node by a further encrypted bit, and the write loop chains 9η multiplications through `hasWritten` for each message. Ciphertext multiplication in the sense of subsection 2.2.1 would exhaust the noise budget long before a round completed, so the authors propose evaluating the tree with the external product, whose noise grows additively along such a chain (Remark 2.29).

Chapter 3

Methodology

OpenFHE does not expose RGSW or the external and internal products through its public API. Running sPAR on BGV therefore meant building the products before the protocol.

This work produced two independent artefacts. The `core` library extends OpenFHE [BAB+22] with RGSW encryption and both products over an RNS gadget, and is independent from sPAR; any OpenFHE project needing RGSW arithmetic can use it as it stands. The `client` and `server` libraries implement the protocol on top of it, reaching `core` only through an abstract interface, so a different backend providing the same API would serve equally well.

Section 3.2 describes the `core` library, from the gadget instantiation and the parallel decomposition through to the two products. Section 3.3 describes our instantiation of sPAR and where it departs from the original specification. The final section describes the harness that produces the measurements of chapter 4 — what is benchmarked, how noise is extracted, and how thread scaling is extrapolated beyond the cores available.

3.1 Code Structure

The source code is published in a git repository [Ang26] as three C++ libraries under `libs/`, each a separate build target with its own public header directory. The `core` library extends OpenFHE with RGSW ciphertexts and related products. The sPAR protocol is split into separate `client` and `server` libraries, so that client and server binaries can be compiled independently.

We highlight that the `core` is entirely independent of sPAR, making it reusable in any OpenFHE project that needs RGSW arithmetic. The `client` and `server` consume the `core` library through an abstract interface.

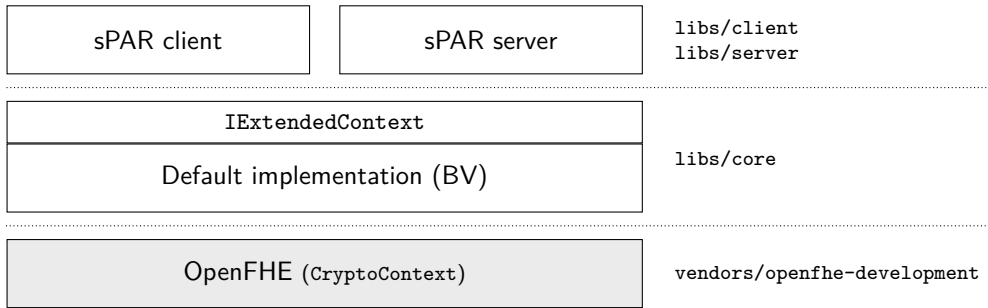


Figure 3.1: Each component depends only on what lies directly beneath it. The sPAR libraries consume the core library solely through the `IExtendedContext` interface.

To ensure our results are replicable, a `make test` command runs the full test suite and noise measurements, while `make bench` runs the benchmarks that produced the measurements reported in chapter 4.

3.2 Implementation Details of the Core Library

The core library, located in the `libs/core` directory of the repository [Ang26], extends OpenFHE [BAB+22] with RGSW functionality that the framework does not itself expose; of which the most important functions are listed in Table 3.1. These are declared by an abstract interface, `IExtendedContext`, allowing alternative implementations to serve as drop-in replacements without requiring user modifications in the future. We also provide a concrete implementation using gadget decomposition of the RNS moduli, as described in section 2.4. Following the convention of [KPZ21; BAB+22], we use *BV* to refer to this technique, after Brakerski and Vaikuntanathan [BV11], and by extension to the implementation built on it.

Table 3.1: Primary functions declared by `IExtendedContext`

Function Signature	Returns
<code>ENCRYPTRGSW(pk, μ)</code>	<code>RGSW(μ)</code>
<code>EVALEXTERNALPRODUCT(x, C)</code>	<code>$x \boxdot C$</code>
<code>EVALINTERNALPRODUCT(A, C)</code>	<code>$A \boxtimes C$</code>

The core library is designed to seamlessly extend OpenFHE, with the aim of making it easy for developers already familiar with OpenFHE to include and use in their own projects. Specifically, OpenFHE works by initializing a *cryptographic context* that holds static parameters, such as the RNS moduli q_i . All homomorphic operations are called through the context object. Our library preserves this pattern: the functions of Table 3.1 are methods on the same context object and operate directly on OpenFHE’s native ciphertext types, so standard and extended operations can be freely interleaved.

Example 3.1. Simple Library Usage

```
// Using standard OpenFHE functions
auto ct = cc->Encrypt(publicKey, m);
auto ct_sqr = cc->EvalMult(ct, ct); // Result is RLWE(m*m)

// Using functions provided by the IExtendedContext interface
auto rgsw_one = cc->EncryptRGSW(publicKey, m); // RGSW(m)
auto ct_ext = cc->EvalExternalProduct(ct, rgsw_one); // RLWE(m*m)
```

Bitwise Operations

Our implementation takes the bit-length $b = \log B$ as a parameter rather than the base B itself. By restricting the base $B = 2^b$ to a power-of-two, we can substitute certain computationally expensive arithmetic operations with hardware-native bitwise operations. They also operate independently on each bit of a machine word without the need for carry propagation, making them highly conducive to SIMD vectorization.¹

Table 3.2: Common bitwise operations

Named Function	Notation	Value
LEFTSHIFT(x, b)	$x \ll b$	$x \cdot 2^b$
RIGHTSHIFT(x, b)	$x \gg b$	$\lfloor x/2^b \rfloor$
AND(x, y)	$x \wedge y$	$\sum_i x_{(2)}[i] \cdot y_{(2)}[i] \cdot 2^i$

Two of these compositions recur often enough to be worth naming. Reduction modulo 2^b is a single mask, and centering a residue into $[-2^{b-1}, 2^{b-1})$ is a mask around an offset; both appear inside the decomposition of the following section.

Algorithm 3.1 CENTER(x, b)

Input: $x, b \in \mathbb{Z}$

Output: $[x]_{2^b} \in [-2^{b-1}, 2^{b-1})$

- 1: mask $\leftarrow$ LEFTSHIFT(1, b) $- 1$
 - 2: half $\leftarrow$ LEFTSHIFT(1, $b - 1$)
 - 3: **return** AND($x +$ half, mask) $-$ half
-

Algorithm 3.2 MODULO(x, b)

Input: $x, b \in \mathbb{Z}$

Output: Unsigned residue $x \bmod 2^b$

- 1: mask $\leftarrow$ LEFTSHIFT(1, b) $- 1$
 - 2: **return** AND(x , mask)
-

¹As an added bonus, replacing traditional branching reduction loops with bitwise masks guarantees constant-time execution, aligning with cryptographic best practices to prevent timing side-channels.

3.2.1 RGSW Encryption

Our implementation instantiates the gadget matrix $G = I_2 \otimes P(1)$ with the double-layered gadget of section 2.4, where the HPS gadget from (2.14) supplies the RNS layer by mapping $R_Q \rightarrow R_{q_0} \times \cdots \times R_{q_{k-1}}$ and the power-of- B gadget $P_g(v) = v \cdot g$ with $g = (1, B, \dots, B^{\ell-1})$ is applied to each limb.

$$P_{\text{BV}}: R_Q \rightarrow R_Q^{k\ell}, \quad P_{\text{BV}}(v) = (P_g \circ P_{\text{HPS}})(v) = P_{\text{HPS}}(v) \otimes g \quad (3.1)$$

Every element of $P_{\text{BV}}(v)$ is a fixed constant times v , so the message can be absorbed into the gadget term of (2.16).

$$C = Z + \mu G = Z + \mu \cdot (I_2 \otimes P_{\text{BV}}(1)) = Z + I_2 \otimes P_{\text{BV}}(\mu) \quad (3.2)$$

Encryption thus reduces to evaluating $P_{\text{BV}}(\mu)$ and adding each of the $k\ell$ entries to two fresh encryptions of zero, once in each component.

Recall from section 2.4 that the j -th component of $P_{\text{HPS}}(\mu)$ reduces to $[\mu]_{q_j}$ modulo its own prime and vanishes modulo every other; so the only non-zero residue of the corresponding entry of $P_{\text{BV}}(\mu)$ is $B^i \cdot [\mu]_{q_j} \bmod q_j = [\mu B^i]_{q_j}$. Writing the $k\ell$ entries in RNS form (2.9), where the i -th row corresponds to residues modulo q_i and each column corresponds to a full RNS polynomial, yields $P_{\text{BV}}(\mu)$ as a *sparse* matrix of $k\ell \times k$ residues rather than a dense vector in $R_Q^{k\ell}$.

$$\begin{bmatrix} [\mu]_{q_0} & [\mu B]_{q_0} & \cdots & [\mu B^{\ell-1}]_{q_0} & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & [\mu]_{q_1} & \cdots & [\mu B^{\ell-1}]_{q_1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 & \cdots & [\mu B^{\ell-1}]_{q_{k-1}} \end{bmatrix}$$

We exploit the sparsity of this structure by discarding the zero residues. Since their contribution to any product is 0, we can simply skip any calculations that would have involved a zero residue. Conveniently, the structure resulting from discarding these elements is exactly the RNS form of $(\mu, \mu B, \dots, \mu B^{\ell-1}) \in R_Q^\ell$. Multiplying μ by a scalar B^i can be done natively in NTT, and the previous derivation does not change if μ is in NTT form, meaning we do not need to perform any NTT or INVERSENTT operations as a result of this gadget.

$$P_{\text{BV}}(\mu)^\top \xrightarrow{\text{flatten}} \begin{bmatrix} [\mu]_{q_0} & \cdots & [\mu B^{\ell-1}]_{q_0} \\ [\mu]_{q_1} & \cdots & [\mu B^{\ell-1}]_{q_1} \\ \vdots & \ddots & \vdots \\ [\mu]_{q_{k-1}} & \cdots & [\mu B^{\ell-1}]_{q_{k-1}} \end{bmatrix} = ([\mu]_{\mathcal{B}_Q}, \dots, [\mu B^{\ell-1}]_{\mathcal{B}_Q})$$

The ciphertext still contains $2k\ell$ rows, each encoding one combination $[\mu B^i]_{q_j}$, but the gadget itself costs nothing to apply. Two benefits follow. There is no transform penalty, since the scaling is a native NTT operation and no conversion is required, and the memory footprint of ENCRYPTRGSW is reduced to the $k\ell$ non-zero residues rather than the $k^2\ell$ of the dense vector. Furthermore, every element of $P_{\text{BV}}(\mu)$ is independent, making half of the $2k\ell$ rows in (3.2) easily parallelizable over $k\ell$ threads.

Algorithm 3.3 Parallel RGSW-BV Encryption

Input: Basis $\mathcal{B}_Q$, decomposition parameter ℓ , and $B^i \bmod q_j \forall i \in [0, \ell), j \in [0, k)$

Input: RNS polynomial $[\mu]_{\mathcal{B}_Q}^{\text{NTT}} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$

Output: RGSW(μ)

- 1: Initialize empty array C with $2k\ell$ slots
 - 2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
 - 3: $(i, j) \leftarrow (r \bmod \ell, \lfloor r/\ell \rfloor)$
 - 4: $mg \leftarrow \text{TOWERMUL}(\mu, j, B^i)$
 - 5: $C_r \leftarrow \text{ENCRYPT}(0) + (mg, 0)$
 - 6: $C_{r+\ell} \leftarrow \text{ENCRYPT}(0) + (0, mg)$
 - 7: **end for**
 - 8: **return** C
-

Precomputed Gadget Powers

In Algorithm 3.3, $\mathcal{B}_Q$, B and ℓ are parameters used to initialize to the cryptographic context, and are thus available to any functions that exist within said context. The values $B^i \bmod q_j$ are calculated and stored ahead of time, when the context is first initialized, and are exposed through a helper function $\text{GETPOWER}(i, j) \leftarrow B^i \bmod q_j$. $\text{TOWERMUL}(\mu, j, B^i)$ extracts the j -th limb from μ , multiplies it by $\text{GETPOWER}(i, j)$, and inserts the product modulo q_j back into the j -th limb.

3.2.2 External Product

The second layer of the gadget from section 2.4 is a decomposition on $g = (1, B, \dots, B^{\ell-1})$, which is essentially digit extraction. Computing the digits as unsigned base- B values puts each d_i in $[0, B)$ and gives quality $\beta = B - 1$.

By Remark 2.29 the noise of an external product grows with β , so it is worth halving. Centering the digits to $d_i \in [-B/2, B/2)$ does exactly that, at $\beta = B/2$. The residues are stored unsigned, so centering is not free: mapping a digit $d_i \geq B/2$ into the negative range subtracts B from it, which must be compensated by a $+1$ carry into d_{i+1} . Standard recoding therefore walks the digits from least to most significant, and that dependency is what prevents the integer from being sliced into independent bit-windows.

To avoid sequential carry propagation, we apply a well-known offset trick by adding the constant Γ (3.3).

$$\Gamma = \frac{B}{2} \cdot \frac{B^\ell - 1}{B - 1} \quad (3.3)$$

The second fraction is the standard mathematical identity for the sum of a finite geometric series, meaning (3.3) represents the g -decomposition of a base- B number γ where every digit is $B/2$.

$$\begin{aligned} \frac{B^\ell - 1}{B - 1} &= \sum_{i=0}^{k-1} B^i = 1 + B + B^2 + \dots + B^{\ell-1} \\ \implies \Gamma &= \sum_{i=0}^{k-1} \frac{B}{2} \cdot B^i = \langle \text{Decomp}(\gamma), g \rangle \end{aligned}$$

To decompose a positive integer $c \in \mathbb{Z}^+$ into signed digits $[d]_B$, we first apply the global offset to compute a shifted, non-negative integer $c' = c + \Gamma$. We then slice c' into intermediate unsigned base- B digits u_i . Because $B = 2^b$ is a power of two, this extraction is performed purely via the independent bitwise Algorithm 3.1, completely bypassing sequential dependencies.

Algorithm 3.4 DECOMPOSE(c, b, ℓ)

Input: Bit-length b (where $B = 2^b$), decomposition parameter ℓ , offset Γ

Input: Integer $c \in \mathbb{Z}$

Output: Array of ℓ signed digits $d = (d_0, d_1, \dots, d_{\ell-1}) \in R_\beta^\ell$ with quality $\beta = B/2$

```

1:  $c' \leftarrow c + \Gamma$  ▷ Shift all digits to positive range  $[0, B]$ 
2:  $\text{mask} \leftarrow \text{LEFTSHIFT}(1, b) - 1$ 
3:  $\text{half} \leftarrow \text{LEFTSHIFT}(1, b - 1)$ 
4: for  $i \leftarrow 0$  to  $\ell - 1$  in parallel do
5:    $s_i \leftarrow \text{RIGHTSHIFT}(c', i \cdot b)$  ▷ Remove smaller digits
6:    $u_i \leftarrow \text{AND}(s_i, \text{mask})$  ▷ Remove larger digits
7:    $d_i \leftarrow \text{AND}(u_i + \text{half}, \text{mask}) - \text{half}$  ▷ Algorithm 3.1
8: end for
9: return  $d = (d_0, d_1, \dots, d_{\ell-1})$ 

```

As with GETPOWER(i, j) from section 3.2.1 the offset Γ is calculated exactly once, when the context is first created.²

²Specifically the bitwise $\Gamma = (1 \ll (b - 1)) \cdot ((2 \ll (b\ell)) - 1) / ((1 \ll b) - 1)$ is used, but this is slightly less human readable than (3.3).

Once we have a working DECOMPOSE function, the external product can be written as Algorithm 3.5. Our implementation is functionally the same, except that it has been split into three stages for better parallelization. First, the $2k$ inverse NTT evaluations required by the decompositions are calculated in parallel, one per tower of each of the two ciphertext elements. Second, the decompose calls are merged into the external product function, and every tower of every digit is sliced, centered and transformed independently, giving $2k\ell \cdot k$ parallel tasks and as many forward NTT calls. Finally, the inner product is accumulated over the $2k$ pairs of ciphertext component and tower, each thread summing its own $2k\ell$ terms with no reduction across threads.

The width of every stage depends on k as well as ℓ , so a parameter set with more RNS primes exposes proportionally more parallelism.

Algorithm 3.5 External Product

Input: RLWE ciphertext $(c_0, c_1) \in R_Q^2$, RGSW ciphertext $C \in R_Q^{2k\ell}$

Output: RLWE ciphertext of product

```

1:  $acc \leftarrow (0, 0)$ 
2:  $d_0 \leftarrow \text{DECOMPOSE}(c_0)$ 
3:  $d_1 \leftarrow \text{DECOMPOSE}(c_1)$ 
4: for  $r \leftarrow [0, k\ell)$  do
5:    $acc[0] \leftarrow acc[0] + d_0[r] \cdot C[r][0] + d_1[r] \cdot C[r + k\ell][0]$ 
6:    $acc[1] \leftarrow acc[1] + d_0[r] \cdot C[r][1] + d_1[r] \cdot C[r + k\ell][1]$ 
7: end for
8: return  $acc$ 

```

3.2.3 Internal Product

The internal product is trivially implemented as repeated external products as per the definition Definition 2.30. Each of the $2k\ell$ rows of the left operand is an independent RLWE ciphertext, so the rows are distributed across threads. Nested parallelism is disabled, meaning the stages inside Algorithm 3.5 collapse to a single thread when it is invoked from here; a standalone external product keeps its own parallelism.

Algorithm 3.6 Internal Product

Input: RGSW ciphertexts $L \in R_Q^{2k\ell}$ and $R \in R_Q^{2k\ell}$

Output: RGSW ciphertext of the product

```

1: Initialize a vector of  $2k\ell$  empty RLWE ciphertexts  $acc$ 
2: for  $i \leftarrow [0, 2k\ell)$  in parallel do
3:    $acc[i] \leftarrow \text{EVAEXTERNALPRODUCT}(acc[i], R)$ 
4: end for
5: return  $acc$ 

```

3.2.4 SIMD Batching

Nothing in the gadget or in either product depends on how the plaintext ring is interpreted, so the implementation inherits the slot packing of Definition 2.12 from OpenFHE unchanged. A caller that builds its plaintexts with `MakePackedPlaintext` rather than `MakeCoefPackedPlaintext` obtains RGSW ciphertexts whose message is a vector of N independent slots, and every external and internal product then acts on all N slots at once. The library selects the matching zero plaintext automatically, so no other change is required.

3.3 Implementation Details of sPAR

Our instantiation of sPAR can be considered the *minimal* version of the original specification [DP25]. What is implemented is the core protocol against a passive adversary, covering setup, encryption, the write step, and partial and final decryption. We do not consider the active threat model(s) of sections 6 and 7, nor the recipient-anonymity layer, which is an ordinary encryption of the message before it enters the system and is therefore orthogonal to the FHE pipeline. Bucket sorting is also not implemented, though this should have a negligible effect as we estimate each bucket can be sorted in parallel in around 100 ms [DP25].

The client-side functions require no new cryptography. The `client::Encrypt` function produces the message ciphertext and the index vectors under the joint key, and `client::Decrypt` produces a partial decryption; both are thin wrappers around the multi-party interface of OpenFHE described in subsection 2.2.2, as is the server’s final decryption `server::Decrypt`. The weight of this section therefore falls on the write step, which is the only part of the protocol that required new machinery.

One consequence of leaving the placement to the clients is visible in what they transmit. Since the server never evaluates the binary tree, clients send the full η -entry indicator vectors rather than the $\log \eta$ encrypted index bits, which costs bandwidth but no computation. We consider bandwidth out of scope, and note only that the tree would otherwise run on the server, where it is parallel across buckets.

3.3.1 Server Write Step

Function names in this chapter follow the `namespace::function` convention of the source, so `server::Write` denotes the implementation of the phase presented as *Server: Write* in subsection 2.5.2. It deviates slightly from [DP25, Algorithm 2]. The first change is structural. The placement of [DP25, Algorithm 2] is split out of the write step and moved to the client, so `server::Write` receives the indicator vectors ready-made and its own body is the second of the paper’s two loops.

The remaining changes concern the homomorphic operations themselves. The paper writes `HomPlacing*` for a generic homomorphic multiplication, which keeps it independent of any scheme; our version is the opposite, and names the specific product used at every step. Typing the operands as in subsection 2.5.3 shows that the external product alone does not cover the circuit. Both `notHasWritten` and $I_{i,k}$ are RGSW ciphertexts carrying state accumulated by earlier iterations, so the two products that form h must be internal products, and only the application of h to the message is external. Where the two products of h are formed, the order of the operands is chosen so that the accumulated flag stays on the left and the freshly received $z_{i,d}$ enters on the right, which is the favourable case of Remark 2.29.

Inverting the flag saves one operation per iteration. The paper multiplies by $(1 - \text{hasWritten})$, whereas tracking the complement `notHasWritten` lets the flag be used directly and subtracted when a slot is claimed. The returned value inverts with it, so the function reports failure rather than success.

Finally, the initial state is not encrypted. The matrices L and I , and the flag `notHasWritten`, are constructed in the correct ciphertext form but with zeroed polynomials in place of a mask, which provides no security on its own. This is harmless because the initial state is public knowledge, and after a single product every entry is an ordinary ciphertext again. Avoiding the encryptions makes the setup considerably cheaper and starts the computation at a marginally lower noise level.

The operands of Algorithm 3.7 are typed accordingly. The indicator entries $z_{i,d}$, the availability entries $I_{i,k}$ and the flag `notHasWritten` are all $\text{RGSW}(m)$ with $m \in \{0,1\}^N$, while the accumulator entries $L_{i,k}$ and the message V_r are $\text{RLWE}(m)$ with $m \in R_t$.

Remark 3.2. From a programming perspective, the choice of $K = 3$ and $D = 3$ is arbitrary. The number of slots per bucket and the number of choices per client are C++ template parameters, also known as generics, and are resolved at compile time rather than carried as runtime values. Our build instantiates them only at $K = D = 3$, and supporting another pair requires nothing beyond adding the corresponding instantiation.

Algorithm 3.7 Server::Write

Input: Reference to state matrices I and L **Input:** Encrypted RLWE value V_r **Input:** Vectors z_0, z_1 and z_2 , where z_d holds η RGSW ciphertexts encrypting a one-hot indicator of the d -th chosen bucket: the entry at that bucket encrypts 1 and every other entry encrypts 0**Output:** Updates the state matrices I and L **Output:** Encrypted flag, 0 if the value was placed and 1 if every candidate slot was occupied

```

1: notHasWritten  $\leftarrow$  ENCRYPTRGSW(1)
2: for  $d \leftarrow \{0, 1, 2\}$  do
3:   for  $k \leftarrow \{0, 1, 2\}$  do
4:     for  $i \leftarrow [0, \eta)$  do
5:        $h \leftarrow (\text{notHasWritten} \boxtimes I_{i,k}) \boxtimes z_{i,d}$ 
6:        $L_{i,k} \leftarrow L_{i,k} + V_r \boxdot h$ 
7:        $I_{i,k} \leftarrow I_{i,k} - h$ 
8:       notHasWritten  $\leftarrow$  notHasWritten  $- h$ 
9:     end for
10:   end for
11: end for
12: return notHasWritten

```

3.4 Experimental Setup

sPAR

Communication between parties is not simulated. In its place a single “all-knowing” orchestrator, holding the joint secret key, performs the setup and one full round. Multi-party FHE is still employed throughout: every user generates its own private key share and a corresponding public share, and the public shares are synthesized into one joint public key.

The orchestrator takes the number of users η as input and gives each user a unique identifier. Every user encrypts its identifier under the joint key and sends it to the server together with three random indices, and once all messages have been received the server performs the write operation on each of them. The setup phase is considered free, as it is static and involves no communication in the simulation.

Measurement and Validation

Timings are collected with Google Benchmark and correctness is checked with GoogleTest. Each benchmark runs 10 repetitions, and every repetition continually repeats the operation within a span of one second. Operations taking longer than a

second therefore yield at least 10 samples and the faster functions yield hundreds, in both cases with a low coefficient of variation.

The two serve different purposes. Benchmarks are optimized for speed and perform no correctness checks, which could otherwise affect the measurement. Each function only needs to run once, since the total time can be extrapolated: client encryption and decryption are run independently by each user and the final decryption is a single aggregation, so the one-shot runtime *is* the runtime, and for the server write only the per-user runtime is of interest. Unit tests are not optimized for speed and instead repeat slow correctness checks at several levels, so that a failure reports not only that an operation failed but *where* and *why*. Because both share the same parameters and code, a passing test empirically validates the correctness of what the benchmarks measure.

Noise Analysis

The testing environment always has access to the secret key, so the error of an RLWE ciphertext is checked directly rather than inferred.

$$x = (c_0, c_1) = (c_1 s + \Delta m + \epsilon, c_1) \quad (3.4)$$

$$\implies \epsilon = c_0 - c_1 s - \Delta m, \quad m = \text{DECRYPT}(x, s) \quad (3.5)$$

An RGSW ciphertext is an intermediate object, so where a result is itself RGSW, as with the internal product or `notHasWritten`, two methods are available. The first row encodes $\mu \cdot 1$, so that row can be extracted on its own and its error checked. Alternatively the external product with `RLWE(1)` can be computed, where that ciphertext is made noiseless so as not to contribute any error of its own. We use the latter, because the first relies on the gadget containing 1, which is traditionally but not necessarily the case.

Multi-Threaded Performance

The main advantage RNS holds over classical single-precision representations is its multi-threading capability. As described above, our implementation allows efficient parallelization over $k\ell$ threads. However, if fewer than $k\ell$ threads are available during benchmarking, we extrapolate the expected performance at $k\ell$ processors by modeling theoretical scaling behavior using Amdahl’s Law [Gus11].

Definition 3.3. Amdahl’s Law A task whose execution is divided into a strictly serial fraction f and a broadly parallelizable fraction $1 - f$ will experience a theoretical speedup over T processors given by:

$$\text{Speedup} = \frac{1}{f + \frac{1-f}{T}} \quad (3.6)$$

Let $E(T)$ denote the total execution time of the algorithm on T threads. Because speedup is defined as $\text{Speedup} = \frac{E(1)}{E(T)}$, we can substitute this ratio into (3.6) and isolate $E(T)$ to form a linear equation:

$$E(T) = E(1)f + \frac{E(1)(1-f)}{T}$$

By mapping this to the variables $y = E(T)$ and $x = \frac{1}{T}$, the relationship resolves into a standard linear model $y = \beta_0 + \beta_1 x$. The intercept $\beta_0 = E(1)f$ represents the fixed serial execution time, while the slope $\beta_1 = E(1)(1-f)$ captures the parallelizable workload scaled by the inverse thread count.

By measuring empirical execution times across a limited set of processor counts ($T = 1, 2, 3, \dots, T_{max}$), we can perform an Ordinary Least Squares (OLS) linear regression on the variable pairs $(\frac{1}{T}, E(T))$. This regression identifies β_0 and β_1 , mathematically separating the serial and parallel workloads. We can then substitute $T = k\ell$ into the fitted model to accurately extrapolate the expected execution time $E(k\ell)$ for a target architecture where $k\ell \gg T_{max}$.

When fitting this regression model, it is critical to exclude measurements taken at complete hardware saturation (i.e., the benchmarking system’s absolute maximum thread count T_{max}). At full capacity, execution time is artificially inflated by external systemic overheads, such as OS thread scheduling, memory bandwidth contention, and thermal throttling. Including this saturated data point warps the linear fit, systematically overestimating the serial fraction f and severely degrading the accuracy of the large-scale extrapolation.

Chapter 4

Results and Discussion

This section presents the noise and performance measurements from the core and sPAR libraries. We also discuss the findings, and suggest some possible optimization.

4.1 Parameters

We found that the parameters suggested in [DP25] results in too much noise for the `Server::Write` function to effectively handle, and are thus not capable of running the full sPAR protocol without decryption failure. Rather than tweaking these parameters until the protocol produces correct decryptions, I instead aim to fully utilize the RNS context by using much larger parameters that enable SIMD batching and far more data transferred per message.

New parameters were thus selected to provide $\lambda \geq 128$ bits of security at the largest possible modulus Q , while supporting SIMD batching, minimizing ℓ so that $k\ell$ is as small as possible, and ensuring sPAR outputs correct results. The RNS moduli are slightly smaller than 64 bits to fit inside 64-bit machine words with a few bits of headroom to prevent overflowing registers.

The resulting parameters, listed in Table 4.1, yield an estimated security parameter of 155.3 bits [APS15] for the underlying RLWE scheme, and serve as the baseline for the benchmarks that follow.

Table 4.1: Base parameters for BV-RNS

Parameter	Value
decomposition (ℓ, B)	$(2, 2^{31})$
number of RNS moduli k	6
size of moduli $\log(q_0)$	60 bits
ciphertext modulus size Q	2^{360}
standard deviation σ	3.19
plaintext modulus t	65537
polynomial degree N	16384
security parameter λ	155.3 bits

4.2 Noise Analysis

This section presents the noise growth of the implemented functions, reported throughout as the max-norm $\|\epsilon\|_\infty = \max_i \{\epsilon_i\}$. The absolute magnitude of the error is of little interest on its own; what matters is how many bits of the modulus it occupies. Every plot in this section is therefore semi-logarithmic with a base-2 y axis, so that a vertical distance reads directly as a number of bits.

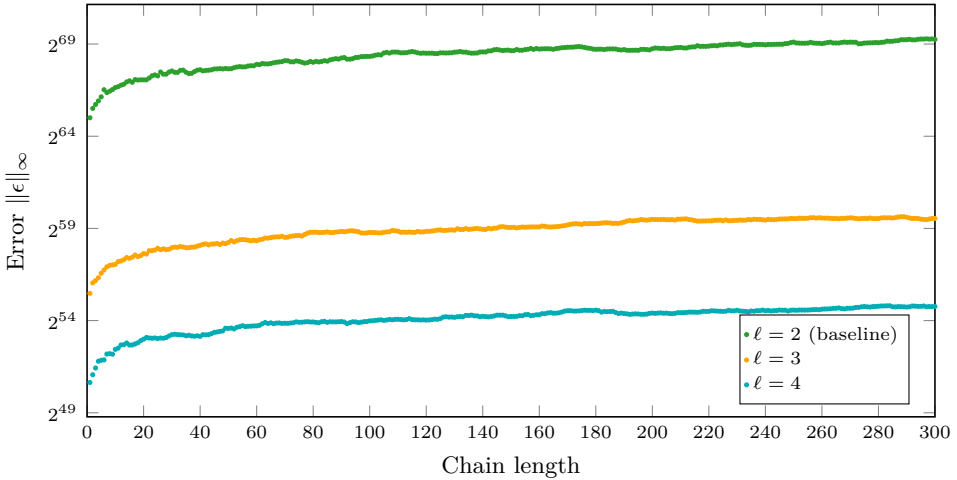
4.2.1 Core Library

For the external product, we apply $x_i = x_{i-1} \boxtimes A_i$ where $x_0 = \text{ENCRYPT}(1)$ using fresh encryptions $A_i = \text{RGSW}(1)$ each iteration.

External Product

Due to the asymmetry discussed in Remark 2.29 making the noise additive instead of multiplicative, we expect to see linear noise growth with repeated applications of the external product as long as the RGSW remains fresh.

Figure 4.1: Accumulated noise from chained external products

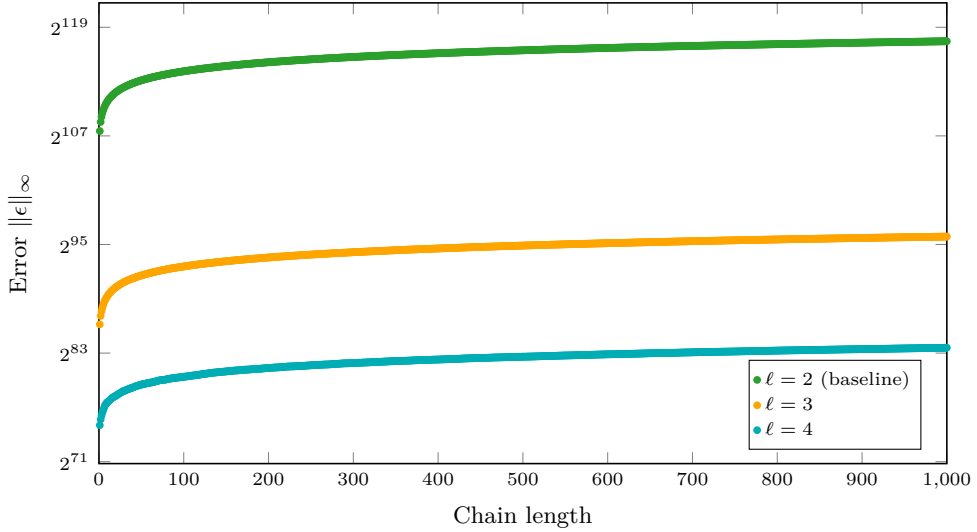


The measured growth is linear in the length of the chain, and the error gains only a few bits across the full chain at every value of ℓ . Increasing ℓ shifts the whole series downwards without changing its slope.

Internal product

For the internal product, we apply $X_i = X_{i-1} \boxtimes A_i$ where $X_0 = \text{RGSW}(1)$ using fresh encryptions $A_i = \text{RGSW}(1)$ each iteration.

Figure 4.2: Accumulated noise from chained internal products



Again we see exactly what we expect; the growth is linear in the length of the chain and again shifts downwards with ℓ . Both the starting error and the amount added per product are substantially larger than for the external product.

4.2.2 sPAR

The protocol was run end-to-end under the multi-party context using OpenFHE’s NOISE_FLOODING_MULTIPARTY mode. With the settings from Table 4.1, we were only able to support $\eta = 4$ users. Increasing ℓ allows more users, but also degrades the performance significantly. Table 4.2 lists the largest error found in the message matrix L once every client has written. Decryption fails once the error reaches $\lfloor Q/2 \rfloor$, which is 359 bits at the parameters of Table 4.1.

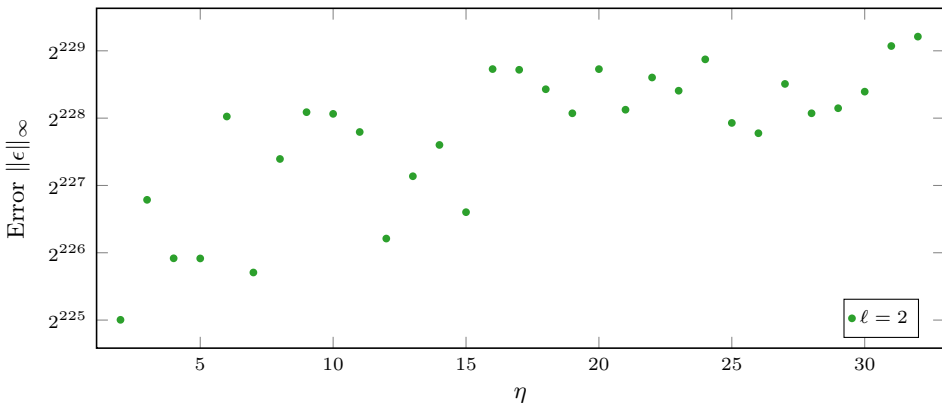
Table 4.2: Largest number of error bits in L after 1 full round.

η	Error (in bits)		
	$\ell = 2$	$\ell = 3$	$\ell = 4$
2	245	221	189
4	358	256	216
8	—	294	*

We were unable to test $\eta \geq 16$ and $\ell = 4$ at $\eta = 8$ both due to the amount of RAM available to the test environment and time constraints (see Table 4.5). However, we estimate the number of users supported at $\ell = 3$ is $\eta < 32$, by considering the average increase in error from $\eta \rightarrow 2\eta$ in the table above.

Initially we suspected this number of users to be artificially low due to the large t . Since the $h \leftarrow I_{i,k} \boxtimes \text{notHasWritten} \boxtimes z_{i,d}$ assignment has only a single fresh ciphertext, $z_{i,d}$, a “dirty” ciphertext necessarily must appear on the right hand side in one of the internal products, breaking the noise asymmetry from Remark 2.29. However, running the same test with $t = 2^8$ only increased the headroom by a few bits.

Figure 4.3: Noise in L after full round.



Testing without multi-party FHE, where every user share a single secret key, reveals the noise growth from the protocol itself is much smaller; to the point where a practically unbounded number of users (on the order of 10^{41}) is supported. This confirms that the large noise in the multi-party setting comes from the noise flooding rather than the protocol itself. The flooding is intrinsic to distributed decryption, where it statistically hides each party’s key share; switching to `FIXED_NOISE_MULTIPARTY`, whose flooding standard deviation is fixed and smaller, should support more users but would weaken that hiding guarantee and require a separate security analysis.

4.3 Performance Benchmarks

The benchmarks were run on an AMD Ryzen 5 5600X CPU (6-core@3.7GHz) with 32Gb RAM in WSL2 Ubuntu 24.04 LTS Linux with the parameters listed in Table 4.1. OpenFHE version 1.5.1 and the code is compiled using GCC v16.1 with all available compiler optimizations, including `-O3` and `-march=native`, except for the AVX512 instruction set (see section 4.4).

4.3.1 Core Library

We present the measured benchmarks in Table 4.3 and extrapolate numbers for $T = 12$ using the method described in section 3.4. The fitted parameters and the resulting estimates are listed in Table 4.4. All three fits give $R^2 \geq 0.994$.

Table 4.3: Mean CPU time in milliseconds spent per operation over T threads

Operation	$T = 1$	$T = 2$	$T = 3$	$T = 4$	$T = 5$	$T = 6$
Encrypt RGSW	123.00	64.80	52.90	41.40	31.80	32.50
External Product	36.90	19.80	14.80	12.20	10.40	10.60
Internal Product	741.00	395.00	281.00	208.00	168.00	176.00

Table 4.4: Fitted serial time β_0 , parallel work β_1 , serial fraction f and extrapolated execution time at $T = 12$

Operation	β_0 (ms)	β_1 (ms)	f	E(12)
Encrypt RGSW	12.44	110.2	10.1%	21.6 ms
External Product	3.75	33.0	10.2%	6.50 ms
Internal Product	34.22	710.3	4.6%	93.4 ms

4.3.2 sPAR

Each function in Table 4.5 except `server::Decrypt` is invoked once per client every round, so the times below are per call. The `client` functions would in practice run in perfect parallel, as each user encrypts and partially decrypts independently.

Table 4.5: Measured computation time per call of the sPAR functions, 6 threads.

η	Encrypt	Write	Partial dec.	Final dec.
2	201.26 ms	8.06798 s	70.3 ms	5.03 ms
4	413.01 ms	15.9965 s	106.28 ms	10.78 ms
8	843.13 ms	33.0402 s	184.07 ms	28.16 ms
16	1648.91 ms	67.368 s	312.84 ms	60.55 ms

4.4 Discussions

The results are slower than anticipated. Even though we are using much larger parameters than the small TFHE parameters suggested in [DP25], the protocol is an order of magnitude slower than what would be practical for most applications. In theory, an arbitrary number of users can be supported, but the parameters that make it possible are too slow for it to be practical in large networks. In the remainder of this section we discuss some potential optimizations that could improve the performance of sPAR.

AVX512 Instruction Set

AVX512 offers true SIMD vectorization, as opposed to the SIMD *packing* of subsection 2.2.1. A single instruction operates on a 512-bit register, letting the CPU process several coefficients of an RNS residue polynomial at once. OpenFHE supports AVX512 acceleration through the Intel HEXL backend, but we were unable to benchmark it because none of the machines available to us have a CPU implementing the instruction set.

Taking full advantage of AVX512 would also motivate revisiting the parameters, since register utilization depends on the sizes of the RNS moduli. A 512-bit register can for example hold eleven 42-bit integers together with one 50-bit integer, or twelve 42-bit integers with some bits left unused. Choosing the moduli to match such a layout, so that $k = 12$ and $Q \approx 2^{512}$, would fill every register completely. Whether this repartitioning of Q leaves the noise behaviour of section 4.2 unchanged is a question we leave open.

4.4.1 SIMD Packing

Throughout this thesis we have emphasized that our implementation supports SIMD packing, but nowhere in sPAR is this utilized. The `server::Write` loop is the most computationally expensive part of the whole protocol. SIMD packing allows the server to perform N rounds simultaneously, making the *per-round* times N times faster, but converting this into higher throughput would require significant changes to the protocol. If N rounds are performed then every client would presumably be encrypting, sending and partially decrypting N times more data, requiring significantly higher bandwidth and RAM on the server.

4.4.2 Theoretical Hybrid Improvement

An alternative to gadget decomposition often employed in RNS key-switching is the *hybrid* technique [GHS12; KPZ21]. A natural question is whether this technique can be employed in the external and internal products.

In hybrid, the idea is to pre-scale a message m by an auxiliary modulus P , where $|P| \approx |Q|/d_{num}$ for some integer $d_{num} \mid k$, resulting in an encryption of $m \cdot P$ in the extended ciphertext modulus QP . After some homomorphic operation(s) we end up with the message $m'P$ and noise $\epsilon' \geq \epsilon$. Dividing the ciphertext by P will result in a ciphertext encrypting m' in modulus Q , scaling down the error by P in the process.

$$\begin{aligned} & (a \cdot s + \Delta m P + \epsilon, a) \pmod{QP} \\ & P^{-1} \cdot [(a \cdot s + \Delta m' P + \epsilon', a) \pmod{QP}] \\ & (a' \cdot s + \Delta m' + \epsilon'/P, a') \pmod{Q} \end{aligned}$$

Reframing this technique in terms of the gadget framework established in section 2.4 is straight forward. If we let $g = P$, the gadget property holds in the extended modulus QP . Using P^{-1} as our decomposition is identical to dividing by P in hybrid.

$$\begin{aligned} \langle x, yP \rangle &= x \cdot y \cdot P \pmod{QP} \\ \langle P^{-1} \cdot x, yP \rangle &= P^{-1} [\langle x, yP \rangle] = x \cdot y \pmod{Q} \end{aligned}$$

This is sufficient to define the external product using the hybrid gadget. This should come as no surprise since key-switching and the external product is essentially the same operation. We must note, however, that this creates an imbalance between the left and right operands, where the right RGSW has rows in the extended QP -modulus space while the left RLWE is defined in the target modulus Q .

Definition 4.1. Hybrid-RGSW An RGSW ciphertext that encodes μP in the extended modulus QP .

$$C_{QP} = Z_{QP} + \mu \begin{pmatrix} P & 0 \\ 0 & P \end{pmatrix} \pmod{QP} \quad (4.1)$$

Thanks to the abstract interface layout of the core library, a second implementation was developed for this thesis that implements the external product. Note that this implementation is technically using the *GHS* technique from [GHS12], which is (by definition) hybrid with $d_{num} = 1$.

Hybrid is very fast in RNS because (1) it uses the fast approximations discussed in subsection 2.3.2 and (2) the decomposition parameter d_{num} is relatively small compared to, for example, the BV gadget with its $k\ell$ parameter.

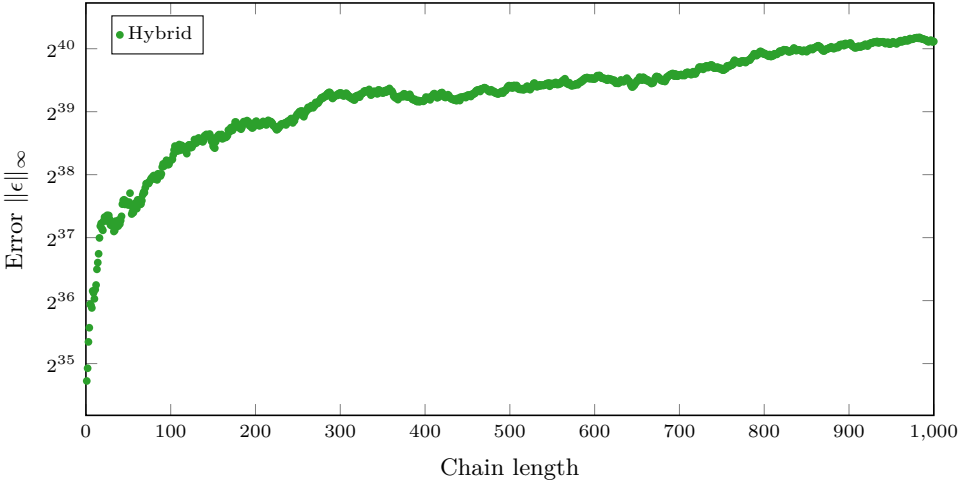
Algorithm 4.1 Hybrid-RGSW Encryption**Input:** Extended basis $\mathcal{B}_P$ **Input:** RNS polynomial $[\mu]_{\mathcal{B}_Q}^{\text{NTT}} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$ **Output:** $\text{RGSW}(\mu \cdot P)$

- 1: Initialize empty array C with $2k\ell$ slots
- 2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
- 3: $mG \leftarrow \text{SCALE}(m, P)$
- 4: $C_r \leftarrow \text{ENCRYPTQP}(0) + (mg, 0)$
- 5: $C_{r+\ell} \leftarrow \text{ENCRYPTQP}(0) + (0, mg)$
- 6: **end for**
- 7: **return** C

Table 4.6: Mean CPU time spent per operation, single-threaded.

Operation	BV	Hybrid
Encrypt RGSW	123.00 ms	20.00 ms
External Product	36.90 ms	18.40 ms

Our testing suggests the hybrid approach not only improves the speed of the external product, but also improves the noise growth properties significantly. In a full implementation this could be tweaked by varying d_{num} , with larger values resulting in even faster speeds at the cost of more noise.

Figure 4.4: Hybrid noise growth in chained external products

Internal Product

One would think that since the internal product is defined as repeated applications of the external product, the hybrid technique would extend to this product as well. It does not. The noise advantage of the hybrid external product comes from the rescaling by P^{-1} , which necessarily maps the result down to modulus Q . Since P is a zero divisor of the ring R_{QP} , the inverse does not exist and division by P is only well-defined as a map $R_{QP} \rightarrow R_Q$. Therefore, a pure “hybrid” internal product cannot exist.

A Compromise

Observe that the pure hybrid internal product only breaks because we require the output to be in a modulus that is a factor of P . If we distinguish between a *left* (regular) and a *right* (hybrid) RGSW, then we can define a *mixed* product that takes a hybrid-RGSW as one of its operands, but outputs a regular RGSW. Importantly, this product would be infinitely chain-able using only hybrid external products.

Corollary 4.2. Mixed Product *Let $\boxdot$ denote the hybrid external product. Let S be an RGSW ciphertext (2.16) with 2ℓ rows and H be a hybrid-RGSW ciphertext (4.1). We define the mixed internal product $\boxtimes$ as*

$$\boxtimes : \text{RGSW} \boxtimes \text{HYBRIDRGSW} \mapsto \text{RGSW} \quad (4.2)$$

$$(A, H) \mapsto A \boxtimes H = \begin{bmatrix} A_0 \boxdot H \\ \vdots \\ A_{2\ell-1} \boxdot H \end{bmatrix} \quad (4.3)$$

and propose that this product is right-chainable wrt. hybrid-RGSW ciphertexts, in the sense that a sequence of hybrid-RGSW ciphertexts H_i can be repeatedly applied as the right operand:

$$(((S \boxtimes H_0) \boxtimes H_1) \dots) \boxtimes H_i \mapsto \text{RGSW}$$

This product has not been implemented in our code, so we can only speculate on its performance. However, it stands to reason that it would outperform the regular internal product in an RNS setting as it uses the faster hybrid external product. In sPAR, specifically, it can be applied to the `server::Write` loop by setting $h \leftarrow (I_{i,k} \boxtimes \text{notHasWritten}) \boxtimes z_{i,d}$ and having the client send z pre-scaled by P and in the QP -modulus space.

To summarize, we believe that if Algorithm 3.7 was rewritten using only external products, the performance would be significantly improved by using the hybrid

technique. Barring that, the protocol might also see a non-negligible speed-up by employing the mixed product when assigning h .

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This thesis set out to answer two questions about sPAR. What does the protocol cost to run, and how far does it scale before it becomes impractical.

On the first question, the measured cost is substantially higher than the original paper anticipates. The cost of a round is dominated by the server's write step, and the write step is dominated by internal products, an operation absent from the cost analysis in [DP25]. A single internal product costs roughly twenty times an external product at our parameters (Table 4.3). The paper's expectation of round times on the order of seconds for a hundred users is not feasible with this instantiation.

On the second question, computation time, not noise, is the binding constraint. A single `server::Write` call grows linearly in η at roughly 4.2 seconds per participant on our hardware, and since the server performs one call per client, the total server time per round grows quadratically. At $\eta = 16$ the server already spends around 18 minutes per round, and extrapolating to $\eta = 100$ gives round times on the order of hours. The noise budget would allow the protocol to run somewhat further, with an estimated $\eta < 32$ at $\ell = 3$ under multi-party noise flooding (Table 4.2), and the protocol itself contributes so little noise that a single-key instantiation supports a practically unbounded number of users. Time renders the protocol impractical well before either limit matters.

The positive result is that sPAR can be instantiated at all from well-established FHE primitives. The protocol runs end-to-end on standard RLWE and RGSW constructions in OpenFHE, with correct decryption under multi-party thresholdized keys. The obstacles we encountered were quantitative rather than structural, which suggests the optimizations below could change the picture materially.

5.2 Future Work

The most direct algorithmic lever is SIMD packing (Definition 2.12) in the write loop [SV11]. Packing lets the server evaluate N rounds simultaneously, dividing the per-round cost of the dominant loop by N , though converting this into throughput requires protocol changes on the client side as discussed in section 4.4.

On the implementation side, OpenFHE’s AVX512 backend was left unmeasured for lack of suitable hardware (section 4.4). Quantifying its concrete speedup, and choosing moduli sized to fill the 512-bit registers, is a natural next step.

Finally, our implementation covers the core of sPAR but not the full protocol, which additionally specifies equivocation protection, bandwidth optimizations, and the sorting of messages into buckets. Completing these components is required before the end-to-end system can be evaluated, but would do little to improve the cost of the protocol.

References

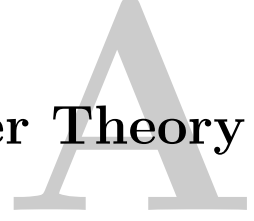
- [Ang26] I. Angelo, *Implementation of (Somewhat) Practical Anonymous Router*, version 1.0.0, Jul. 2026. [Online]. Available: <https://github.com/ImreAngelo/sPAR>.
- [AP14] J. Alperin-Sheriff and C. Peikert, *Faster bootstrapping with polynomial error*, Cryptology ePrint Archive, Paper 2014/094, 2014. [Online]. Available: <https://eprint.iacr.org/2014/094>.
- [APS15] M. R. Albrecht, R. Player, and S. Scott, *On the concrete hardness of learning with errors*, Cryptology ePrint Archive, Paper 2015/046, 2015. [Online]. Available: <https://eprint.iacr.org/2015/046>.
- [BAB+22] A. A. Badawi, A. Alexandru, *et al.*, *OpenFHE: Open-source fully homomorphic encryption library*, Cryptology ePrint Archive, Paper 2022/915, 2022. [Online]. Available: <https://eprint.iacr.org/2022/915>.
- [BEHZ16] J.-C. Bajard, J. Eynard, *et al.*, *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: <https://eprint.iacr.org/2016/510>.
- [BGV11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *Fully homomorphic encryption without bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. [Online]. Available: <https://eprint.iacr.org/2011/277>.
- [Bra12] Z. Brakerski, *Fully homomorphic encryption without modulus switching from classical GapSVP*, Cryptology ePrint Archive, Paper 2012/078, 2012. [Online]. Available: <https://eprint.iacr.org/2012/078>.
- [BV11] Z. Brakerski and V. Vaikuntanathan, *Efficient fully homomorphic encryption from (standard) LWE*, Cryptology ePrint Archive, Paper 2011/344, 2011. [Online]. Available: <https://eprint.iacr.org/2011/344>.
- [BV13] Z. Brakerski and V. Vaikuntanathan, *Lattice-based FHE as secure as PKE*, Cryptology ePrint Archive, Paper 2013/541, 2013. [Online]. Available: <https://eprint.iacr.org/2013/541>.
- [CCR19] H. Chen, I. Chillotti, and L. Ren, “Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe”, in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 345–360. [Online]. Available: <https://doi.org/10.1145/3319535.3354226>.

- [CDPP22] K. Cong, D. Das, *et al.*, “Sortingat: Efficient private decision tree evaluation via homomorphic encryption and transciphering”, in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’22, Los Angeles, CA, USA: Association for Computing Machinery, 2022, pp. 563–577. [Online]. Available: <https://doi.org/10.1145/3548606.3560702>.
- [CGGI16] I. Chillotti, N. Gama, *et al.*, *Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds*, Cryptology ePrint Archive, Paper 2016/870, 2016. [Online]. Available: <https://eprint.iacr.org/2016/870>.
- [CGGI18] I. Chillotti, N. Gama, *et al.*, *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: <https://eprint.iacr.org/2018/421>.
- [CKKS16] J. H. Cheon, A. Kim, *et al.*, *Homomorphic encryption for arithmetic of approximate numbers*, Cryptology ePrint Archive, Paper 2016/421, 2016. [Online]. Available: <https://eprint.iacr.org/2016/421>.
- [DM14] L. Ducas and D. Micciancio, *FHEW: Bootstrapping homomorphic encryption in less than a second*, Cryptology ePrint Archive, Paper 2014/816, 2014. [Online]. Available: <https://eprint.iacr.org/2014/816>.
- [DP25] D. Das and J. Park, *sPAR: (somewhat) practical anonymous router*, Cryptology ePrint Archive, Paper 2025/860, 2025. [Online]. Available: <https://eprint.iacr.org/2025/860>.
- [FV12] J. Fan and F. Vercauteren, *Somewhat practical fully homomorphic encryption*, Cryptology ePrint Archive, Paper 2012/144, 2012. [Online]. Available: <https://eprint.iacr.org/2012/144>.
- [Gen09] C. Gentry, “A fully homomorphic encryption scheme”, Ph.D. dissertation, Stanford University, 2009. [Online]. Available: <https://crypto.stanford.edu/craig>.
- [GHS12] C. Gentry, S. Halevi, and N. P. Smart, *Homomorphic evaluation of the AES circuit*, Cryptology ePrint Archive, Paper 2012/099, 2012. [Online]. Available: <https://eprint.iacr.org/2012/099>.
- [GSW13] C. Gentry, A. Sahai, and B. Waters, *Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based*, Cryptology ePrint Archive, Paper 2013/340, 2013. [Online]. Available: <https://eprint.iacr.org/2013/340>.
- [Gus11] J. L. Gustafson, “Amdahl’s law”, in *Encyclopedia of Parallel Computing*, D. Padua, Ed. Boston, MA: Springer US, 2011, pp. 53–60. [Online]. Available: https://doi.org/10.1007/978-0-387-09766-4_77.
- [HPS18] S. Halevi, Y. Polyakov, and V. Shoup, *An improved RNS variant of the BFV homomorphic encryption scheme*, Cryptology ePrint Archive, Paper 2018/117, 2018. [Online]. Available: <https://eprint.iacr.org/2018/117>.
- [KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: <https://eprint.iacr.org/2021/204>.

- [LPR12] V. Lyubashevsky, C. Peikert, and O. Regev, *On ideal lattices and learning with errors over rings*, Cryptology ePrint Archive, Paper 2012/230, 2012. [Online]. Available: <https://eprint.iacr.org/2012/230>.
- [LZ22] Z. Liang and Y. Zhao, *Number theoretic transform and its applications in lattice-based cryptosystems: A survey*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.13546>.
- [RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, “On data banks and privacy homomorphisms”, in *Foundations of Secure Computation*, 1978, pp. 169–180.
- [Reg09] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography”, *Journal of the ACM*, vol. 56, no. 6, 34:1–34:40, 2009.
- [SEV+15] Y. Sun, A. Edmundson, *et al.*, “RAPTOR: Routing attacks on privacy in tor”, in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun>.
- [SV11] N. P. Smart and F. Vercauteren, *Fully homomorphic SIMD operations*, Cryptology ePrint Archive, Paper 2011/133, 2011. [Online]. Available: <https://eprint.iacr.org/2011/133>.
- [SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: <https://eprint.iacr.org/2021/435>.

Appendix

Number Theory



A.1 The Chinese Remainder Theorem

Definition A.1. Chinese Remainder Theorem Let $q_0, q_1, \dots, q_{k-1}$ be a set of k pairwise coprime positive integers, meaning $\gcd(q_i, q_j) = 1$ for all $i \neq j$. For any given sequence of arbitrary integers $x_0, x_1, \dots, x_{k-1}$, there exists a unique solution x modulo the product of these moduli that satisfies the following system of simultaneous congruences:

$$\begin{aligned} x &\equiv x_0 \pmod{q_0} \\ &\vdots \\ x &\equiv x_{k-1} \pmod{q_{k-1}} \end{aligned}$$

Let $Q = \prod_{i=0}^{k-1} q_i$, $\hat{q}_i = \frac{Q}{q_i}$ and $\hat{q}_i^{-1}$ be the inverse s.t. $\hat{q}_i^{-1} \hat{q}_i \equiv 1 \pmod{q_i}$

$$\text{Then CRT states } x = \sum_{i=0}^{k-1} x_i \cdot \hat{q}_i \cdot \hat{q}_i^{-1} \pmod{Q} \quad (\text{A.1})$$

$$x = \langle \{x_i\}_{i=0}^{k-1}, \{\hat{q}_i \cdot \hat{q}_i^{-1}\}_{i=0}^{k-1} \rangle \pmod{Q} \quad (\text{A.2})$$

A.2 The Number Theoretic Transform

The efficiency of modern ring-based homomorphic encryption relies strictly on substituting high-complexity polynomial arithmetic with low-complexity scalar operations. This appendix formalizes the algebraic mechanics of the Number Theoretic Transform (NTT) over the negacyclic polynomial ring $R_{q_i} = \mathbb{Z}_{q_i}[X]/(X^N + 1)$, which dictates the behavior of a single RNS tower.

Definition A.2. Principal Roots of Unity Let N be a power of two and q be an N -transformable modulus such that $q \equiv 1 \pmod{2N}$. The algebraic field $\mathbb{Z}_q$ is

guaranteed to contain a **primitive $2N$ -th root of unity**, denoted ψ , which strictly satisfies $\psi^{2N} \equiv 1 \pmod{q}$ and $\psi^k \not\equiv 1 \pmod{q}$ for all $0 < k < 2N$. Consequently, it holds that $\psi^N \equiv -1 \pmod{q}$.

Definition A.3. Forward Negacyclic NTT Let $a \in R_q$ be a polynomial defined by its coefficient vector $(c_0, c_1, \dots, c_{N-1})$. The roots of the cyclotomic polynomial $X^N + 1 = 0$ in $\mathbb{Z}_q$ are explicitly the odd powers of ψ : $\{\psi^1, \psi^3, \dots, \psi^{2N-1}\}$.

The **Forward NTT** is the linear evaluation mapping of $a(X)$ exclusively at these specific roots. The transformed vector $\hat{a} = \text{NTT}(a)$ is defined component-wise for $j \in \{0, 1, \dots, N-1\}$ as:

$$\hat{a}_j = a(\psi^{2j+1}) = \sum_{i=0}^{N-1} c_i \psi^{(2j+1)i} \pmod{q} \quad (\text{A.3})$$

Definition A.4. Inverse Negacyclic NTT Because the evaluation points are distinct roots of unity, the forward mapping is a strict algebraic isomorphism and is thereby unconditionally invertible. Given the transformed vector $\hat{a}$, the **Inverse NTT** (INTT) reconstructs the original polynomial coefficients c_i for $i \in \{0, 1, \dots, N-1\}$ via the orthogonal properties of these roots. The inverse mapping is formulated as:

$$c_i = \text{INTT}(\hat{a})_i = N^{-1} \psi^{-i} \sum_{j=0}^{N-1} \hat{a}_j \psi^{-2ij} \pmod{q} \quad (\text{A.4})$$

where N^{-1} denotes the modular multiplicative inverse of N in $\mathbb{Z}_q$.

Theorem A.5. Negacyclic Convolution Theorem Let $a, b \in R_q$. The coefficient vector of their polynomial product $c = a \cdot b \pmod{X^N + 1}$ is canonically isomorphic to the Hadamard (element-wise) product of their respective NTT forms. Specifically:

$$c = \text{INTT}(\text{NTT}(a) \odot \text{NTT}(b)) \quad (\text{A.5})$$

By exploiting the symmetry of roots of unity via fast decimation algorithms, functionally identical to the Fast Fourier Transform utilizing the Cooley-Tukey butterfly structure, the computational evaluation of both the forward and inverse NTT is mathematically reduced from a standard complexity of $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ [LZ22]. This precise algorithmic reduction forms the absolute baseline requirement for practical latency in RNS-variant homomorphic cryptography.

Appendix

Noise Asymmetry in RGSW Products

Remark 2.29 notes an asymmetry . Here is the full derivation

$$\begin{aligned}
 x \cdot C &= G^{-1}(x)(Z + \mu G) \\
 G^{-1}(x) \cdot Z + \mu(G^{-1}(x) \cdot G) \\
 G^{-1}(x)Z + \mu(x + \epsilon_g)
 \end{aligned}$$

where $\|G^{-1}\|_{\infty} \leq \beta$ is inherited

$$\text{Using (2.21): } G^{-1}(x)Z = [D(x_0)|D(x_1)] \begin{pmatrix} \text{RLWE}(0) \\ \vdots \\ \text{RLWE}(0) \end{pmatrix}$$

$$\text{Let } D_i = G^{-1}(x)[i] \text{ and } Z_i = (a_i \cdot s + \epsilon_i, a_i)$$

$$\begin{aligned} \Rightarrow G^{-1}(x) \cdot Z &= \left(\sum_{i=0}^{2\ell-1} a_i D_i s + \epsilon_i D_i, \sum_{i=0}^{2\ell-1} a_i D_i \right) \\ &\left(\left[\sum_{i=0}^{2\ell-1} a_i D_i \right] s + \sum_{i=0}^{2\ell-1} \epsilon_i D_i, \sum_{i=0}^{2\ell-1} a_i D_i \right) \end{aligned}$$

$$\text{Let } a = \sum_{i=0}^{2\ell-1} a_i D_i \text{ and } \epsilon = \sum_{i=0}^{2\ell-1} \epsilon_i, \text{ and recall } \|D(x_i)\|_\infty \leq \beta$$

$$\begin{aligned} (a \cdot s + \sum_{i=0}^{2\ell-1} \epsilon_i D_i, a) &\leq (a \cdot s + \sum_{i=0}^{2\ell-1} \epsilon_i \beta, a) \\ \therefore G^{-1}(x) \cdot Z &\leq (a \cdot s + \beta \epsilon, a) \end{aligned}$$

So clearly, the error sources are:

- $\mu \epsilon_g$ which is 0 if the gadget is exact
- $\beta \sum e_z$ the sum of

xxx